

Mixed Models: a Primer

2026-06-15

```
pckgs <- c("tidyverse", "scales", "gt", "broom", "lme4", "nlme", "knitr", "patchwork",
  "broom.mixed", "lmerTest", "emmeans")
donotshow <- purrr::map(pckgs, library, character.only = TRUE)
| Warning: package 'ggplot2' was built under R version 4.5.3
| -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
| v dplyr      1.1.4      v readr      2.1.5
| v forcats    1.0.1      v stringr    1.5.2
| v ggplot2     4.0.3      v tibble     3.3.0
| v lubridate   1.9.4      v tidyr      1.3.1
| v purrr      1.1.0
| -- Conflicts ----- tidyverse_conflicts() --
| x dplyr::filter() masks stats::filter()
| x dplyr::lag()     masks stats::lag()
| i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to be
|
| Attaching package: 'scales'
|
|
| The following object is masked from 'package:purrr':
|
|   discard
|
|
| The following object is masked from 'package:readr':
```

```
| col_factor
|
| Loading required package: Matrix
|
| Attaching package: 'Matrix'
|
| The following objects are masked from 'package:tidyr':
|
|   expand, pack, unpack
|
| Attaching package: 'nlme'
|
| The following object is masked from 'package:lme4':
|
|   lmList
|
| The following object is masked from 'package:dplyr':
|
|   collapse
| Warning: package 'patchwork' was built under R version 4.5.3
| Attaching package: 'lmerTest'
|
| The following object is masked from 'package:lme4':
|
|   lmer
|
| The following object is masked from 'package:stats':
|
|   step
|
| Welcome to emmeans.
| Caution: You lose important information if you filter this package's results.
| See '? untidy'
select <- dplyr::select
```

```
options(pillar.sigfig = 4, pillar.neg = "zigzag")
```

```
tidy_gt <- function(model, type = c("summary", "anova"), decimals = 3, ...) {
  type <- match.arg(type)

  if (type == "summary") {
    out <- tidy(model, ...)
    gt(out) |>
      fmt_number(columns = c(estimate, std.error, statistic), decimals = decimals) |>
      fmt_scientific(columns = p.value) |>
      cols_label(term = "Term", estimate = "Estimate", std.error = "Std. Error",
        statistic = "t value", p.value = "p value") |>
      tab_options(table.font.size = 14)

  } else {
    out <- tidy(anova(model), ...)
    gt(out) |>
      fmt_number(columns = c(df, sumsq, meansq, statistic), decimals = decimals) |>
      fmt_scientific(columns = p.value) |>
      cols_label(term = "Term", df = "df", sumsq = "Sum Sq", meansq = "Mean Sq",
        statistic = "F value", p.value = "p value") |>
      tab_options(table.font.size = 14)

  }
}

glance_gt <- function(model, decimals = 3, ...) {
  glance(model, ...) |>
    gt() |>
    fmt_number(decimals = decimals) |>
    cols_label(r.squared = html("R2"), adj.r.squared = html("adj. R2"), sigma = " ",
      AIC = "AIC", BIC = "BIC", logLik = "logLik") |>
    tab_options(table.font.size = 14)
}

tidy_mixed_gt <- function(model, ...) {
  broom.mixed::tidy(model, ...) |>
    mutate(across(where(is.numeric), \(x) round(x, 4))) |>
    gt() |>
    tab_options(table.font.size = 13)
}

glance_mixed_gt <- function(model) {
  broom.mixed::glance(model) |>

```

```
mutate(across(where(is.numeric), \(x) round(x, 3))) |>
gt() |>
tab_options(table.font.size = 13)
}
```

```
theme_lmm <- function(base_size = 11) {
  theme_minimal(base_size = base_size) + theme(plot.title = element_text(size = 16,
    face = "bold", margin = margin(b = 6)), plot.subtitle = element_text(size = 14,
    color = "#555", margin = margin(b = 10)), axis.title = element_text(size = 14),
    axis.text.x = element_text(size = 10), axis.text.y = element_text(size = 10),
    axis.line = element_line(color = "black"), panel.grid.minor = element_blank(),
    panel.grid.major = element_line(color = "#efefef"), legend.position = "bottom",
    legend.title = element_text(size = 14), legend.text = element_text(size = 14),
    strip.text = element_text(face = "bold", size = 12))
}
theme_set(theme_lmm())
```

i Vitality turns Mixed

- Aim: develop intuition for mixed models
- Focus on
 - equations (building blocks)
 - visualizations

```
load("zenzi.RData")
```

Context

Long History

- 1950: Henderson equations define mixed model
- 1982: Laird and Ware model allows modeling growth trajectories
- 1990: SAS PROC MIXED makes it possible for us, now in all major software

Added Value

- MANOVA made flexible and efficient

- Extends regression; modeling dependencies

```
group_grays_9 <- gray.colors(9, start = 0.2, end = 0.8)

col1 <- "#039"
col2 <- "#f60"

group_colors_9 <- colorRampPalette(c(col1, "#99664D", col2))(9)
```

```
set.seed(123)
n <- 64
mu_true <- 15
sigma <- sqrt(5)
scores <- rnorm(n, mean = mu_true, sd = sigma)
id <- seq_along(scores)
dta_base <- data.frame(id = id, score = scores)
```

```
n1 <- n%/%2
n2 <- n - n1
dta_groups <- dta_base |>
  mutate(group = factor(c(rep("group 1", n1), rep("group 2", n2))))

tmp <- dta_groups
for (it in 1:3) {
  tmp <- tmp |>
    mutate(tmp = case_when(group == "group 1" ~ ifelse(score < 15, 15, score),
      group == "group 2" ~ ifelse(score > 15, 15, score)))
  tmp <- tmp |>
    mutate(group = factor(case_when(score == max(tmp) ~ "group 2", score == min(tmp) ~
      "group 1", TRUE ~ group)))
}
dta_groups <- tmp |>
  arrange(group, id) |>
  mutate(id = 1:n())
dta_groups <- dta_groups |>
  mutate(group = recode(group, `group 1` = "control", `group 2` = "treatment"))
```

```
dta_cont <- dta_base |>
  full_join(dta_base |>
    arrange(score) |>
    mutate(id2 = 1:n())) |>
  mutate(idx = id + id2) |>
```

```
arrange(idx) |>
mutate(idx = 1:n(), age = 25 + idx + runif(n(), -0.5, 0.5)) |>
select(id, score, age)
| Joining with `by = join_by(id, score)`
```

```
dta_dich <- dta_cont |>
mutate(age_d = factor(age > 60, labels = c("age 60", "age >60"))) |>
arrange(age)
```

```
dta_int <- dta_cont |>
mutate(group = factor(id > 32, labels = c("0", "1"), levels = c(FALSE, TRUE))) |>
mutate(group = recode(group, `0` = "control", `1` = "treatment"))
```

```
set.seed(2025)
n_per_group <- 30
T_r <- 5
dta_rct <- do.call(rbind, lapply(1:n_per_group, function(i) {
  do.call(rbind, lapply(0:1, function(trt) {
    u0 <- rnorm(1, 0, 6)
    u1 <- rnorm(1, 0, 0.4)
    time <- 0:(T_r - 1)
    y <- 50 + 0 * trt + (1 + 2 * trt) * time + u0 + u1 * time + rnorm(T_r, 0,
      3)
    data.frame(id = paste(trt, i, sep = "_"), trt = factor(trt, levels = c("0",
      "1"), labels = c("C", "T")), time = time, y = y)
  })))
}))
```

Regression

Working Example

Let's assume a SQUARE test exists

- with test results between 0 and 20
- performed on patients
- who have particular characteristics (e.g., age, ...)
- under particular conditions (e.g., treated or not)

Aim: understand the conditions (e.g., treatment) under which test results are higher/lower, accounting for age

- by relating data (test results ~ outcome) to conditions and characteristics (predictors) → columns
- using information offered by each of the patients (replications ~ sample size) → rows

Equation-wise:

$$\text{square test result}_{id} = \beta_0 + \beta_1 * \text{Treatment}_{id} + \beta_2 * \text{Age}_{id} + \varepsilon_{id}, \quad \text{for } id = 1, \dots, n,$$

More generically:

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \varepsilon_i, \quad \text{for } i = 1, \dots, n.$$

Regression (and ANOVA)

- is modeling variance (e.g., in y_i)
- turning it into a conditional mean (e.g., in $\beta_0 + \beta_1 * \text{Treatment}_{id} + \beta_2 * \text{Age}_{id}$)
- by minimizing squared error (e.g., of ε_i^2).

Regression Model: Mean and Variance

Regression as swiss knife.

A regression model with only an intercept extracts

- mean (conditional on nothing else)
- variance (of the residuals around the mean)
- standard error (uncertainty around the estimated mean)

$$y_i = \beta_0 + \varepsilon_i \quad \text{with} \quad \varepsilon_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2).$$

```
ybar <- mean(dta_base$score)

s <- sd(scores)
se <- s/sqrt(n)

alpha_ci <- 0.05
t_crit <- qt(1 - alpha_ci/2, df = n - 1)
```

```
ci_lower <- ybar - t_crit * se
ci_upper <- ybar + t_crit * se

ggplot(dta_base, aes(x = id, y = score)) + geom_rect(xmin = 0.5, xmax = n + 0.5,
  ymin = mu_true - sigma, ymax = mu_true + sigma, fill = "gray80", alpha = 0.3,
  inherit.aes = FALSE) + geom_rect(xmin = 0.5, xmax = n + 0.5, ymin = ci_lower,
  ymax = ci_upper, fill = "gray60", alpha = 0.25, inherit.aes = FALSE) + annotate("rect",
  xmin = n + 1.5, xmax = n + 2, ymin = ybar - se, ymax = ybar + se, fill = "#f60",
  alpha = 0.8) + geom_point(size = 2, color = "#333333", alpha = 0.9) + geom_segment(aes(x
  xend = id, y = score, yend = ybar), color = "#666666", alpha = 0.4, linewidth = 1) +
  geom_hline(yintercept = ybar, color = "#1f78b4", linewidth = 2, linetype = "dashed") +
  annotate("text", x = n - 1, y = ybar + 0.5, label = "SE", color = "#f60", hjust = -1.1,
    size = 5) + annotate("text", x = 2, y = ybar + 0.8, label = paste0("Intercept = mean
  round(ybar, 2)), color = "#039", hjust = 0, size = 5) + annotate("segment", x = 3,
  xend = 4, y = mu_true + sigma, yend = mu_true + sigma, color = "gray70", linewidth = 2,
  alpha = 0.6) + annotate("text", x = 5, y = mu_true + sigma, label = "±1 SD",
  hjust = 0, color = "gray30", size = 3.5) + labs(x = "Patient", y = "Result",
  title = "Intercept-only model") + coord_cartesian(ylim = c(10, 20))
```

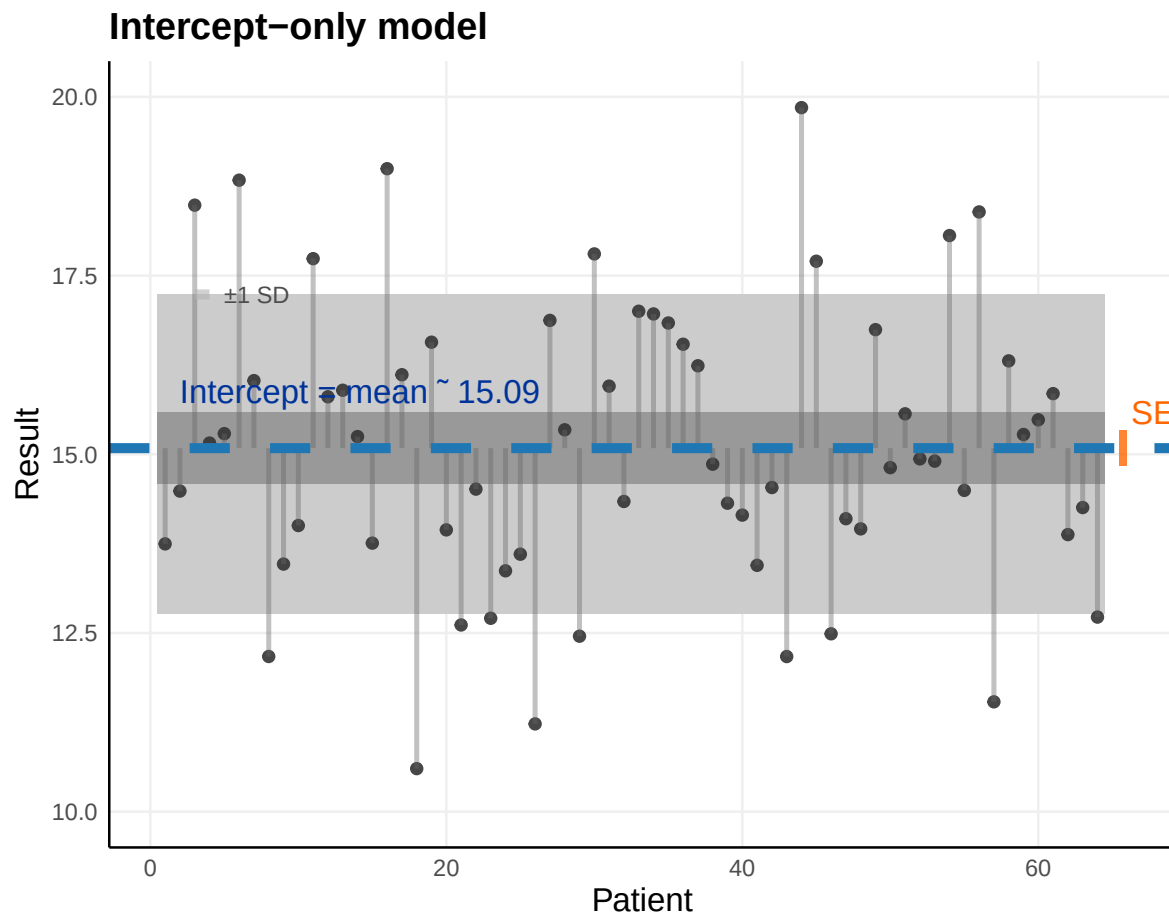



Figure 1: Intercept-only model

Patients on x-axis, observations on y-axis

- line represents average
- greyed band +/- 1 standard deviation

Fitted intercept is the mean,
or average SQUARE test result

$$\hat{\beta}_0 = \bar{y}$$

Residuals are difference data - model based expectation.

$$e_i = y_i - \bar{y}$$

Term	Estimate	Std. Error	t value	p value
(Intercept)	15.086	0.251	60.221	2.11×10^{-57}

R ²	adj. R ²	σ	statistic	p.value	df	logLik	AIC	BIC	deviance	df.residual	nobs
0.000	0.000	2.004	NA	NA	NA	-134.801	273.601	277.919	253.034	63.000	64.000

No variance explained, with total variance σ^2 .

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (y_i - \mu)^2$$

Predicts the same value for every case.

$$SE(\hat{\beta}_0) = \frac{\sigma}{\sqrt{n}}$$

The standard error (SE; estimation) is a function of the standard deviation (sd; data).

```
lm_b0 <- lm(scores ~ 1, data = dta_base)
```

```
tidy_gt(lm_b0)
```

```
glance_gt(lm_b0)
```

Two Groups; t-test look-alike

A regression model with an intercept and a treatment effect (slope, dichotomous variable Treatment vs. Control)

- treatment and control specific means
- difference between treatment and control
- standard error of difference

The regression model is:

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \quad \text{for } i = 1, \dots, n,$$

With

$$x_i = \begin{cases} 0 & \text{for control,} \\ 1 & \text{for treatment.} \end{cases}$$

And the usual distributional assumptions.

$$\varepsilon_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2).$$

```
# use dta_groups, col1, col2
grp_stats <- dta_groups |>
  group_by(group) |>
  summarise(mean = mean(score), sd = sd(score), n = n(), se = sd(score)/sqrt(n()),
    .groups = "drop") |>
  mutate(alpha_ci = 0.05, t_crit = qt(1 - alpha_ci/2, df = n - 1), ci_lower = mean -
    t_crit * se, ci_upper = mean + t_crit * se)

pltdta_groups <- dta_groups |>
  left_join(grp_stats |>
    select(group, mean), by = "group")

ggplot(pltdta_groups, aes(x = id, y = score, color = group, fill = group)) + geom_point(size =
  alpha = 0.9) + scale_color_manual(values = c(control = col1, treatment = col2)) +
  geom_segment(data = pltdta_groups, aes(x = id, xend = id, y = score, yend = mean),
    color = "#666666", alpha = 0.4, linewidth = 1) + geom_hline(data = grp_stats,
  aes(yintercept = mean, color = group), linewidth = 2.5, linetype = "dashed") +
  geom_text(data = data.frame(x = 1.2, y = grp_stats$mean[1] + 0.7, label = paste0("control mean ",
    round(grp_stats$mean[1], 2)), group = "Control"), aes(x = x, y = y, label = label,
    color = group), hjust = 0, size = 4, inherit.aes = FALSE) + geom_text(data = data.frame(x = 1.2,
  y = grp_stats$mean[2] + 0.7, label = paste0("treatment mean ", round(grp_stats$mean[2],
    2)), group = "Treatment"), aes(x = x, y = y, label = label, color = group),
  hjust = 0, size = 4, inherit.aes = FALSE) + geom_rect(data = data.frame(xmin = c(0.5,
  n1 + 0.75), xmax = c(n1 + 0.25, n + 0.5), ymin = grp_stats$mean - grp_stats$sd,
  ymax = grp_stats$mean + grp_stats$sd, group_fill = c(col1, col2)), aes(xmin = xmin,
  xmax = xmax, ymin = ymin, ymax = ymax, fill = group_fill), alpha = 0.3, inherit.aes = FALSE) +
  scale_fill_manual(values = c(control = col1, treatment = col2), guide = "none") +
  annotate("text", x = 8, y = grp_stats$mean[1] - grp_stats$sd[1], label = "±1 SD (G1)",
    hjust = 0, color = "gray30", size = 3.5) + annotate("text", x = 40, y = grp_stats$mean[2] -
  grp_stats$sd[2], label = "±1 SD (G2)", hjust = 0, color = "gray30", size = 3.5) +
  labs(x = "Patient", y = "Result", title = "Two-group model", color = "") + coord_cartesian(ylim = c(
  20)) + guides(color = guide_legend(override.aes = list(size = 3)))
```

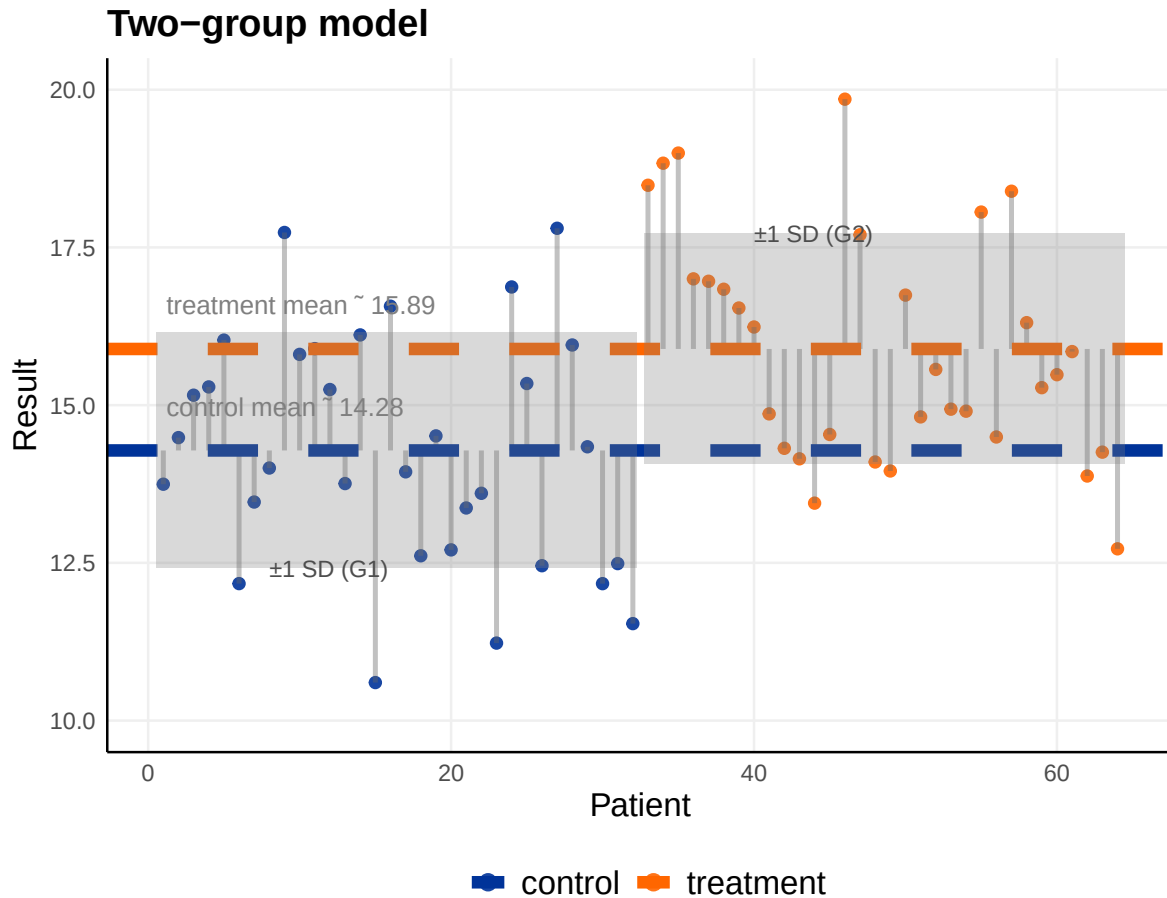


Figure 2: Two group-specific means

Patients on x-axis, observations on y-axis

- line represents group specific means
- greyed band ± 1 group specific standard deviation

Fitted intercept is the mean for group coded as 0,
fitted slope is the difference between the two groups offer estimated group means (conditional means)

$$\hat{\beta}_0 = \bar{y}_{x=0}$$

$$\hat{\beta}_1 = \bar{y}_{x=1} - \bar{y}_{x=0}$$

Residuals are still difference data - model based expectation.

Term	Estimate	Std. Error	t value	p value
(Intercept)	14.282	0.327	43.728	2.62×10^{-48}
group.treatment	1.609	0.462	3.483	9.15×10^{-4}

R ²	adj. R ²	σ	statistic	p.value	df	logLik	AIC	BIC	deviance	df.residual	nobs
0.164	0.150	1.848	12.131	0.001	1.000	-129.082	264.165	270.641	211.627	62.000	64.000

$$e_i = y_i - \hat{y}_i$$

Notice: residuals are defined over all observations, not per condition.

The standard error (SE; estimation) is a function of the group specific standard deviations (sd; data) and sample sizes.

About 16.36 % of the variance is now explained.

Predicts the same value for every case within a group.

```
lm_groups <- lm(score ~ group, data = dta_groups)
```

```
tidy_gt(lm_groups)
```

```
glance_gt(lm_groups)
```

Continuous Variable

A regression model with an intercept and a continuous predictor (slope)

- predictor specific means
- difference for 1 unit extra on the predictor
- standard error of the slope

The regression model is exactly the same as before:

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \quad \text{for } i = 1, \dots, n,$$

with the usual distributional assumption:

$$\varepsilon_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2).$$

```
# use dta_cont
lm_cont <- lm(score ~ age, data = dta_cont)

cont_stats <- dta_cont |>
  summarise(mean_score = mean(score), sd_score = sd(score), n = n(), se_mean = sd(score)/sqrt(n),
    .groups = "drop")

pltdta_cont <- dta_cont |>
  mutate(fitted = predict(lm_cont), resid = score - fitted)

col_cont <- col1

ggplot(pltdta_cont, aes(x = age, y = score)) + geom_point(size = 2, alpha = 0.9,
  color = col_cont) + geom_segment(data = pltdta_cont, aes(x = age, xend = age,
  y = score, yend = fitted), color = "gray60", alpha = 0.4, linewidth = 1) + geom_smooth(
  se = TRUE, color = col_cont, linewidth = 1.5, alpha = 0.2, fullrange = TRUE) +
  geom_hline(data = cont_stats, aes(yintercept = mean_score), color = "gray60",
    linewidth = 2, linetype = "dashed", alpha = 0.8) + geom_rect(data = data.frame(xmin =
  0.5, xmax = max(pltdta_cont$age) + 1, ymin = cont_stats$mean_score - cont_stats$se_mean,
  ymax = cont_stats$mean_score + cont_stats$se_mean), aes(xmin = xmin, xmax = xmax,
  ymin = ymin, ymax = ymax), fill = col2, alpha = 0.4, color = "black", linewidth = 0.5,
  inherit.aes = FALSE) + geom_rect(data = data.frame(xmin = min(pltdta_cont$age) -
  1, xmax = max(pltdta_cont$age) + 1, ymin = cont_stats$mean_score - cont_stats$sd_score,
  ymax = cont_stats$mean_score + cont_stats$sd_score), aes(xmin = xmin, xmax = xmax,
  ymin = ymin, ymax = ymax), fill = "gray70", alpha = 0.15, inherit.aes = FALSE) +
  annotate("text", x = max(pltdta_cont$age) - 1, y = cont_stats$mean_score + 0.3,
    label = "SE", color = col2, hjust = -0.1, size = 5) + annotate("text", x = min(pltdta_cont$age) +
  1, y = cont_stats$mean_score + cont_stats$sd_score, label = "+1 SD", hjust = 0,
  color = "gray30", size = 3.5) + annotate("text", x = mean(pltdta_cont$age), y = max(pltdta_cont$score) +
  0.8, label = paste0("Linear trend: score  ", round(coef(lm_cont)[1], 2), " + ",
  round(coef(lm_cont)[2], 3), " × age"), size = 4.5) + labs(x = "Age (continuous predictor)",
  y = "Score (outcome)", title = "Continuous predictor model") + coord_cartesian(ylim = c(
  20))
| `geom_smooth()` using formula = 'y ~ x'
```

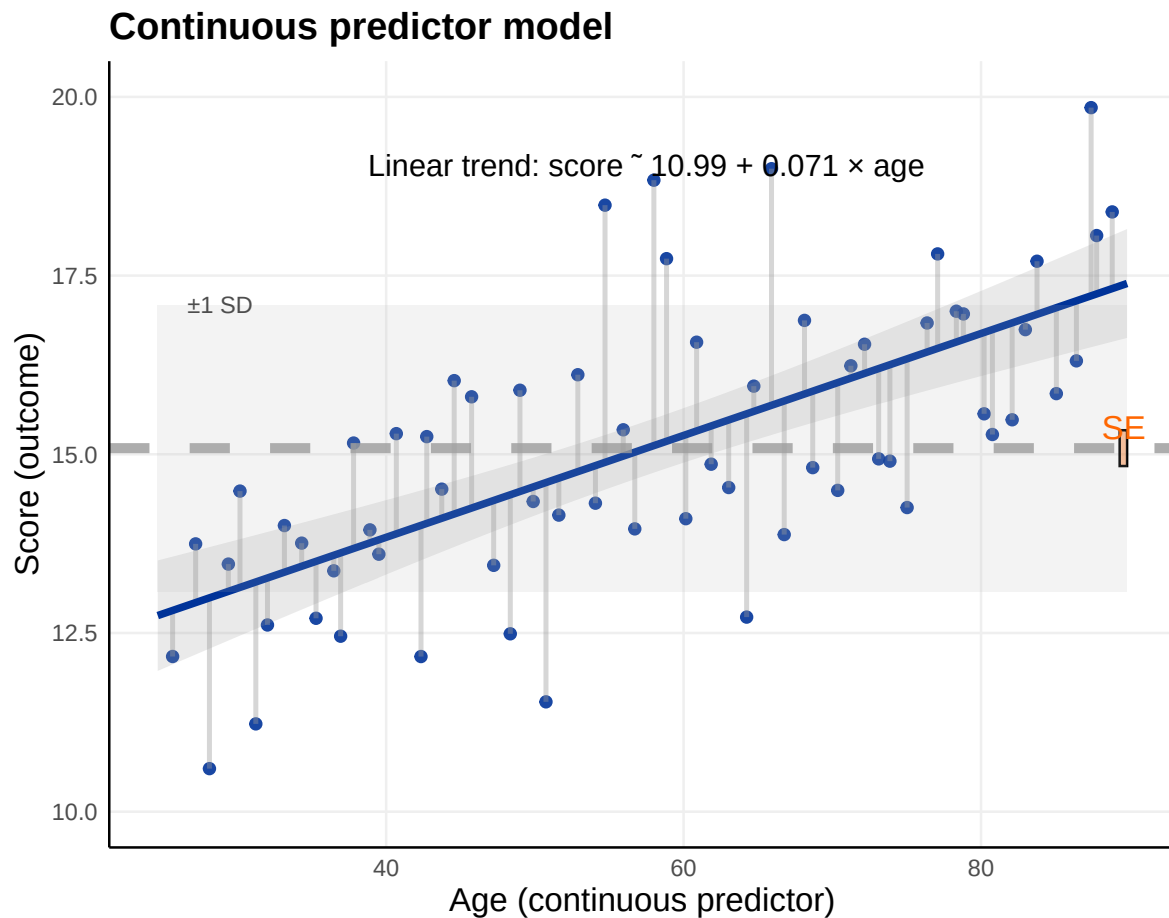


Figure 3: Predictor value specific means

Note, the same 64 scores are used, but treated as continuous predictor x_i .

Predictor x_i on **x-axis** (age), observations on y-axis

- line represents **linear trend**
- greyed band +/- 1 **overall** standard deviation

Fitted intercept is the mean at $x_i = 0$

Fitted slope is increase per unit x_i ,

Note: center your x_i around another value,
change the intercept.

$$\hat{\beta}_0 = \mathbb{E}[Y \mid X = 0]$$

Term	Estimate	Std. Error	t value	p value
(Intercept)	10.987	0.618	17.770	5.15×10^{-26}
age	0.071	0.010	6.963	2.49×10^{-9}

R ²	adj. R ²	σ	statistic	p.value	df	logLik	AIC	BIC	deviance	df.residual	nobs
0.439	0.430	1.513	48.484	0.000	1.000	-116.313	238.626	245.103	141.995	62.000	64.000

$$\hat{\beta}_1 = \frac{\Delta \mathbb{E}[Y]}{\Delta X}$$

Residuals are difference
data - model based expectation.

$$e_i = y_i - \hat{y}_i$$

$$SE(\hat{\beta}_1) = \sqrt{\frac{sd^2}{n \cdot \text{Var}(X)}}$$

The standard error (SE; estimation) is a function of the **residual variance** relative to predictor spread.

```
lm_cont <- lm(score ~ age, data = dta_cont)
```

```
tidy_gt(lm_cont)
```

```
glance_gt(lm_cont)
```

Dichotomized continuous predictor

Same model !

A regression model with an intercept and a continuous predictor that is dichotomized into two groups.


```
# use dta_dich, col1, col2

grp_stats <- dta_dich |>
  group_by(age_d) |>
  summarise(mean = mean(score), sd = sd(score), n = n(), se = sd(score)/sqrt(n()),
    .groups = "drop") |>
  mutate(alpha_ci = 0.05, t_crit = qt(1 - alpha_ci/2, df = n - 1), ci_lower = mean -
    t_crit * se, ci_upper = mean + t_crit * se)

n1 <- sum(dta_dich$age_d == "age 60")
n2 <- sum(dta_dich$age_d == "age >60")

pltdta_dich <- dta_dich |>
  left_join(grp_stats |>
    select(age_d, mean), by = "age_d")

ggplot(pltdta_dich, aes(x = age, y = score, color = age_d, fill = age_d)) + geom_point(size = 100,
  alpha = 0.9) + scale_color_manual(values = c(`age 60` = col1, `age >60` = col2)) +
  geom_segment(data = pltdta_dich, aes(x = age, xend = age, y = score, yend = mean),
    color = "#666666", alpha = 0.4, linewidth = 1) + geom_hline(data = grp_stats,
  aes(yintercept = mean, color = age_d), linewidth = 2.5, linetype = "dashed") +
  geom_text(data = data.frame(x = mean(pltdta_dich$age[pltdta_dich$age_d == "age 60"]),
    y = grp_stats$mean[1] + 0.7, label = paste0("age 60 mean  ", round(grp_stats$mean[1],
    2)), age_d = "age 60"), aes(x = x, y = y, label = label, color = age_d),
    hjust = 0, size = 4, inherit.aes = FALSE) + geom_text(data = data.frame(x = mean(pltdta_dich$age[pltdta_dich$age_d == "age >60"]),
    y = grp_stats$mean[2] + 0.7, label = paste0("age >60 mean  ",
    round(grp_stats$mean[2], 2)), age_d = "age >60"), aes(x = x, y = y, label = label,
    color = age_d), hjust = 0, size = 4, inherit.aes = FALSE) + geom_rect(data = data.frame(x = min(pltdta_dich$age[pltdta_dich$age_d == "age >60"]),
    xmax = c(max(pltdta_dich$age[pltdta_dich$age_d == "age 60"]),
    max(pltdta_dich$age[pltdta_dich$age_d == "age >60"])), ymin = grp_stats$mean[1] -
    grp_stats$sd, ymax = grp_stats$mean + grp_stats$sd, group_fill = c(col1, col2)),
  aes(xmin = xmin, xmax = xmax, ymin = ymin, ymax = ymax, fill = group_fill), alpha = 0.3,
  inherit.aes = FALSE) + scale_fill_manual(values = c(`age 60` = col1, `age >60` = col2),
  guide = "none") + annotate("text", x = mean(pltdta_dich$age[pltdta_dich$age_d ==
  "age 60"]) - 1.5, y = grp_stats$mean[1] + grp_stats$sd[1], label = "+1 SD ( 60)",
  hjust = 1, color = "gray30", size = 3.5) + annotate("segment", x = mean(pltdta_dich$age[pltdta_dich$age_d == "age >60"]) - 1,
  xend = mean(pltdta_dich$age[pltdta_dich$age_d == "age >60"]) -
  0.5, y = grp_stats$mean[2] + grp_stats$sd[2], yend = grp_stats$mean[2] + grp_stats$sd[2],
  color = col2, linewidth = 3, alpha = 0.6) + annotate("text", x = mean(pltdta_dich$age[pltdta_dich$age_d == "age >60"]) - 1.5,
  y = grp_stats$mean[2] + grp_stats$sd[2], label = "+1 SD (>60)",
  hjust = 1, color = "gray30", size = 3.5) + labs(x = "Age", y = "Score", title = "Dichotomous Regression") + coord_cartesian(ylim = c(10, 20)) + guides(color = guide_legend(override.aes = FALSE))
```

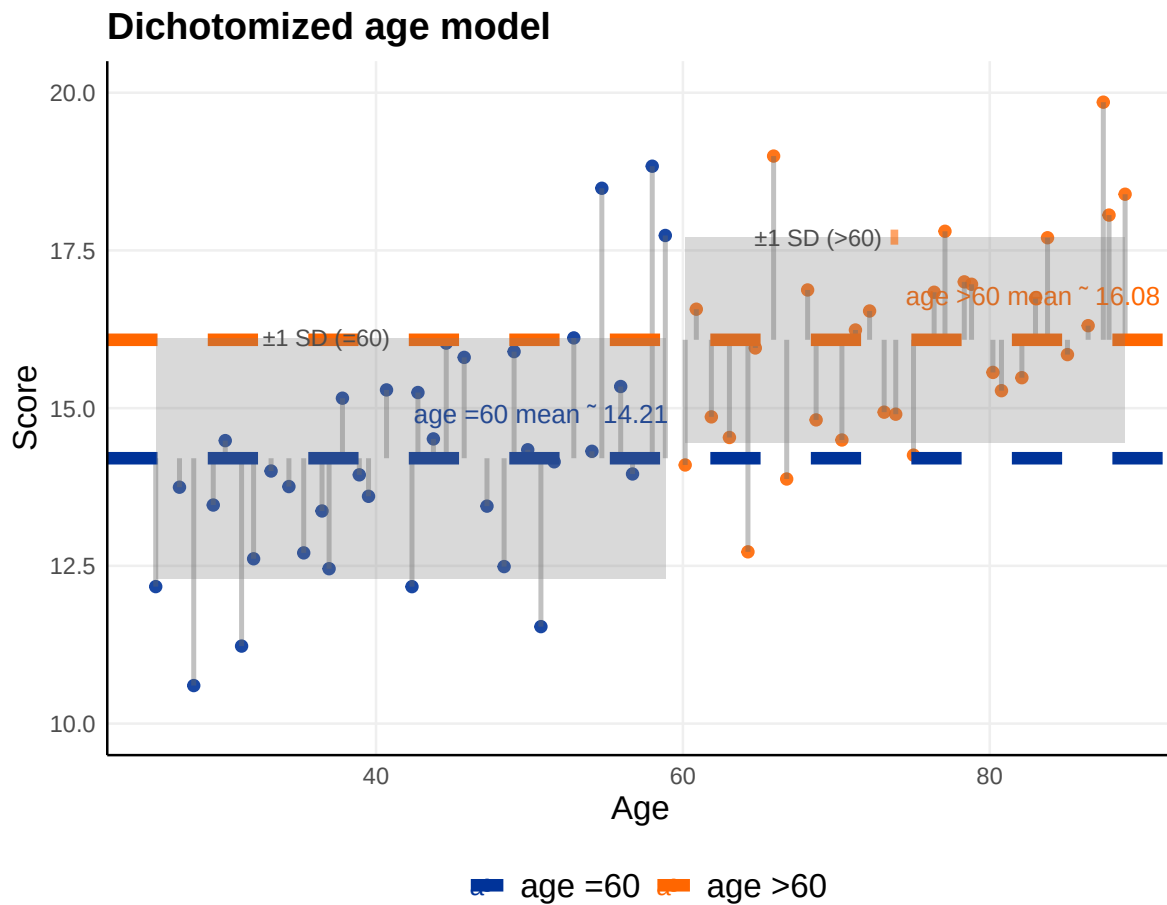


Figure 4: Dichotomized age groups

Predictor X_i on **x-axis** (age), observations on y-axis

- line represents group specific average
- greyed band +/- 1 group specific standard deviation

Same model, but

- residual variance increases
- explained variance decreases
- residuals are clustered, not independent

violation of the distributional assumption:

$$\varepsilon_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2).$$

Term	Estimate	Std. Error	t value	p value
(Intercept)	14.206	0.306	46.487	6.59×10^{-50}
age_dage >60	1.877	0.446	4.206	8.51×10^{-5}

R ²	adj. R ²	σ	statistic	p.value	df	logLik	AIC	BIC	deviance	df.residual	nobs
0.222	0.209	1.782	17.692	0.000	1.000	-126.767	259.535	266.011	196.858	62.000	64.000

```
lm_dich <- lm(score ~ age_d, data = dta_dich)
```

```
tidy_gt(lm_dich)
```

```
glance_gt(lm_dich)
```

Multiple Regression; combining information

A regression model with an intercept, a group difference, and a continuous predictor

- predictor specific means
- that can differ depending on other predictors
- with standard errors for each slope

The regression model starts being interesting:

$$y_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \beta_3 x_{1i} x_{2i} + \varepsilon_i, \quad \text{for } i = 1, \dots, n,$$

with the usual distributional assumption:

$$\varepsilon_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2).$$

Interaction

Interactions would allow effects to be condition dependent.

- Age effect differs per group.
- Group difference are age dependent.

```
# use dta_int

lm_int <- lm(score ~ age * group, data = dta_int)

int_stats_overall <- dta_int |>
  summarise(mean_score = mean(score), sd_score = sd(score), n = n(), se_mean = sd_score/sqrt(n),
    .groups = "drop")

pltdta_int <- dta_int |>
  mutate(fitted = predict(lm_int), resid = score - fitted)

col_g0 <- col1
col_g1 <- col2

ggplot(pltdta_int, aes(x = age, y = score, color = group)) + geom_point(size = 2,
  alpha = 0.9) + geom_smooth(method = "lm", se = TRUE, linewidth = 1.5, alpha = 0.2,
  fullrange = TRUE) + geom_hline(data = int_stats_overall, aes(yintercept = mean_score),
  color = "gray60", linewidth = 2, linetype = "dashed", alpha = 0.8) + geom_rect(data = int_stats_overall,
  1, xmax = max(pltdta_int$age) + 1, ymin = int_stats_overall$mean_score - int_stats_overall$sd_score,
  ymax = int_stats_overall$mean_score + int_stats_overall$sd_score), aes(xmin = 0,
  xmax = xmax, ymin = ymin, ymax = ymax), fill = "gray70", alpha = 0.15, inherit.aes = FALSE) +
  annotate("text", x = mean(pltdta_int$age), y = max(pltdta_int$score) - 0.4, label = paste0(
    round(coef(lm_int)[["(Intercept)"]], 2), " + ", round(coef(lm_int)[["age"]],
    3), " × age"), color = col_g0, size = 4) + annotate("text", x = mean(pltdta_int$age),
  y = max(pltdta_int$score) - 1, label = paste0("treatment: score  ", round(coef(lm_int)[["group:treatment"]],
    3), " ", round(coef(lm_int)[["group:treatment"]], 3), ") + (", round(coef(lm_int)[["age:group:treatment"]],
    3), " ", round(coef(lm_int)[["age:group:treatment"]], 3), ") × age"), color = col_g1,
  size = 4) + labs(x = "Age (continuous predictor)", y = "Score (outcome)", color = "Group") +
  title = "Continuous predictor with group interaction") + coord_cartesian(ylim = c(10,
  20)) + scale_color_manual(values = c(control = col_g0, treatment = col_g1)) +
  theme(legend.position = "none")
| `geom_smooth()` using formula = 'y ~ x'
```

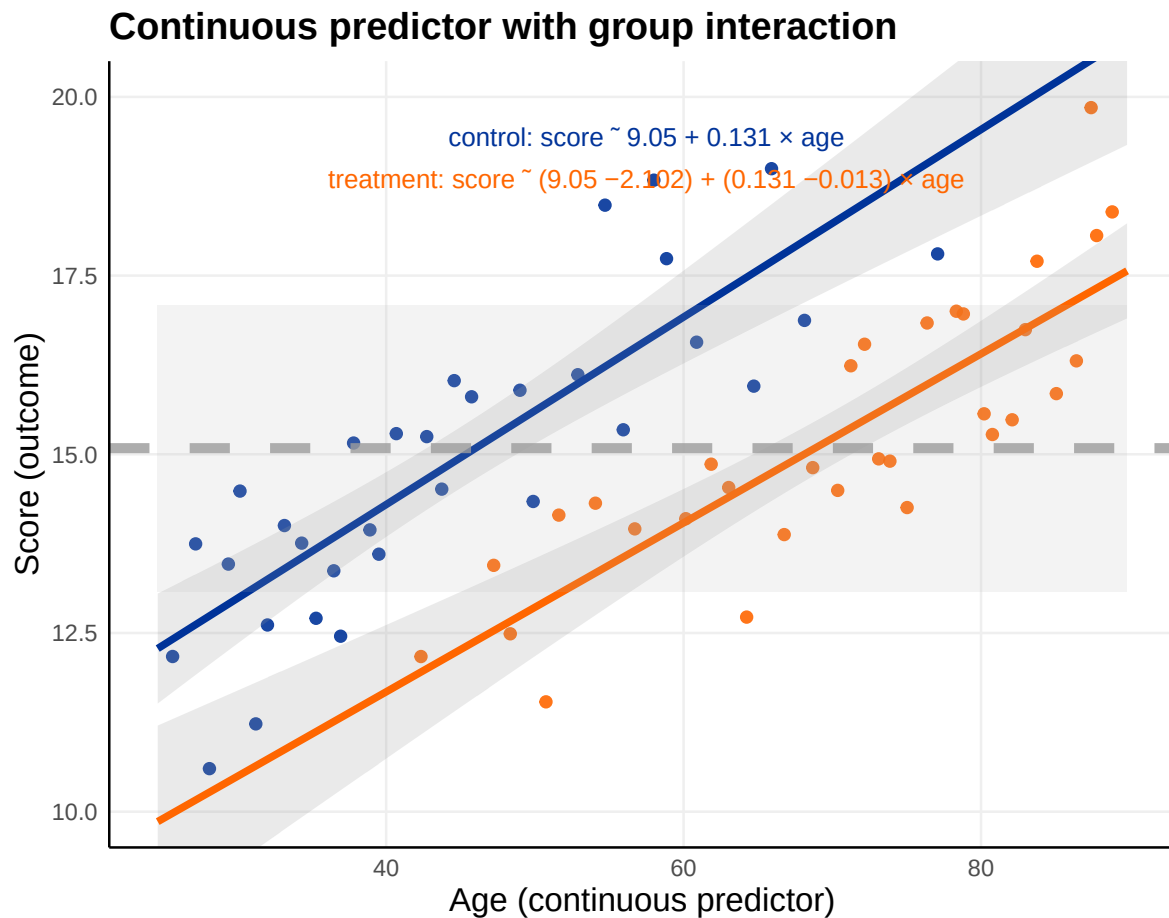


Figure 5: Interacting Predictors

Predictor x_{2i} on **x-axis** (age), observations on y-axis

- line represents group specific **linear trend**
- greyed band ± 1 group specific standard deviation

```
lm_int <- lm(score ~ age * group, data = dta_int)
```

```
tidy_gt(lm_int)
```

In this case, no evidence for an interaction.

Term	Estimate	Std. Error	t value	p value
(Intercept)	9.050	0.684	13.227	1.94×10^{-19}
age	0.131	0.015	8.942	1.24×10^{-12}
grouptreatment	-2.102	1.266	-1.660	1.02×10^{-1}
age:grouptreatment	-0.013	0.021	-0.620	5.38×10^{-1}

R ²	adj. R ²	σ	statistic	p.value	df	logLik	AIC	BIC	deviance	df.residual	nobs
0.707	0.692	1.112	48.215	0.000	3.000	-95.539	201.077	211.872	74.187	60.000	64.000

```
glance_gt(lm_int)
```

Main effects

Simplifying the model and its interpretation, the interaction is removed.

When age is considered jointly with group.

```
# use dta_int

lm_int_main <- lm(score ~ age + group, data = dta_int)

int_stats_overall <- dta_int |>
  summarise(mean_score = mean(score), sd_score = sd(score), n = n(), se_mean = sd_score/sqrt(n),
    .groups = "drop")

pltdta_int_main <- dta_int |>
  mutate(fitted = predict(lm_int_main), resid = score - fitted)

col_g0 <- col1
col_g1 <- col2

ggplot(pltdta_int_main, aes(x = age, y = score, color = group)) + geom_point(size = 2,
  alpha = 0.9) + geom_smooth(method = "lm", se = TRUE, linewidth = 1.5, alpha = 0.2,
  fullrange = TRUE) + geom_hline(data = int_stats_overall, aes(yintercept = mean_score),
  color = "gray60", linewidth = 2, linetype = "dashed", alpha = 0.8) + geom_rect(data = dta_int,
  1, xmax = max(pltdta_int_main$age) + 1, ymin = int_stats_overall$mean_score -
  int_stats_overall$sd_score, ymax = int_stats_overall$mean_score + int_stats_overall$sd_score,
  aes(xmin = xmin, xmax = xmax, ymin = ymin, ymax = ymax), fill = "gray70", alpha = 0.15,
```

```
inherit.aes = FALSE) + annotate("text", x = mean(pltdta_int_main$age), y = max(pltdta_int_main$score) - 0.4, label = paste0("group 0: score  ", round(coef(lm_int_main)[["(Intercept)"]], 2), " + ", round(coef(lm_int_main)[["age"]], 3), " x age"), color = col_g0, size = 4) + annotate("text", x = mean(pltdta_int_main$age), y = max(pltdta_int_main$score) - 1, label = paste0("group 1: score  (", round(coef(lm_int_main)[["(Intercept)"]], 3), " ", round(coef(lm_int_main)[["group1treatment"]], 3), ") + ", round(coef(lm_int_main)[["age"]], 3), " x age"), color = col_g1, size = 4) + labs(x = "Age (continuous predictor)", y = "Score (outcome)", color = "Group", title = "Continuous predictor without group interaction", coord_cartesian(ylim = c(10, 20)) + scale_color_manual(values = c(control = col_g0, treatment = col_g1)) + theme(legend.position = "none")
| `geom_smooth()` using formula = 'y ~ x'
```

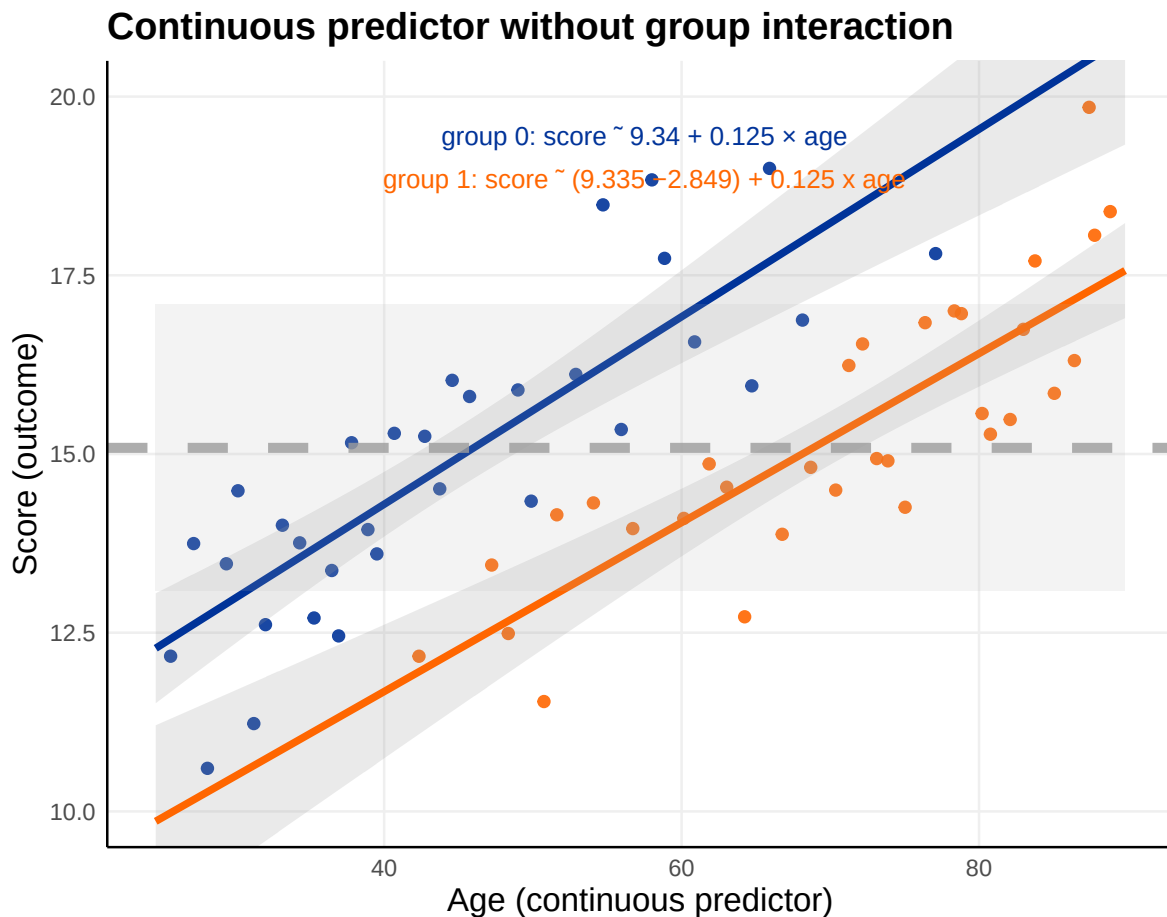


Figure 6: Non-interacting Predictors

Predictor x_{2i} on **x-axis** (age), observations on y-axis

Term	Estimate	Std. Error	t value	p value
(Intercept)	9.335	0.504	18.527	9.82×10^{-27}
age	0.125	0.010	12.005	1.04×10^{-17}
grouptreatment	-2.849	0.384	-7.417	4.44×10^{-10}

R ²	adj. R ²	σ	statistic	p.value	df	logLik	AIC	BIC	deviance	df.residual	nobs
0.705	0.695	1.106	72.866	0.000	2.000	-95.743	199.486	208.122	74.662	61.000	64.000

- line represents group specific **linear trend**
- greyed band +/- 1 group specific standard deviation

```
lm_int_main <- lm(score ~ age + group, data = dta_int)
```

```
tidy_gt(lm_int_main)
```

There is a very clear group effect. In this case, no evidence for an interaction.

```
glance_gt(lm_int_main)
```

Ignoring a relevant predictor

With only group in the model, ignoring age.

```
# use dta_int

lm_int_ign <- lm(score ~ group, data = dta_int)

int_stats_overall <- dta_int |>
  summarise(mean_score = mean(score), sd_score = sd(score), n = n(), se_mean = sd_score/sqrt(n),
    .groups = "drop")

pltdta_int_ign <- dta_int |>
  mutate(fitted = predict(lm_int_ign), resid = score - fitted)

col_g0 <- col1
col_g1 <- col2
```



```
ggplot(pltdta_int_ign, aes(x = group, y = score, color = group)) + geom_jitter(size = 2,
  alpha = 0.9, width = 0.2) + geom_smooth(method = "lm", se = TRUE, linewidth = 1.5,
  alpha = 0.2, fullrange = TRUE) + geom_hline(data = int_stats_overall, aes(yintercept = m,
  color = "gray60", linewidth = 2, linetype = "dashed", alpha = 0.8) + labs(x = "Groups (categorical predictor)",
  y = "Score (outcome)", color = "Group", title = "Ignoring continuous predictor") +
  coord_cartesian(ylim = c(10, 20)) + scale_color_manual(values = c(control = col_g0,
  treatment = col_g1)) + theme(legend.position = "none")
| `geom_smooth()` using formula = 'y ~ x'
```

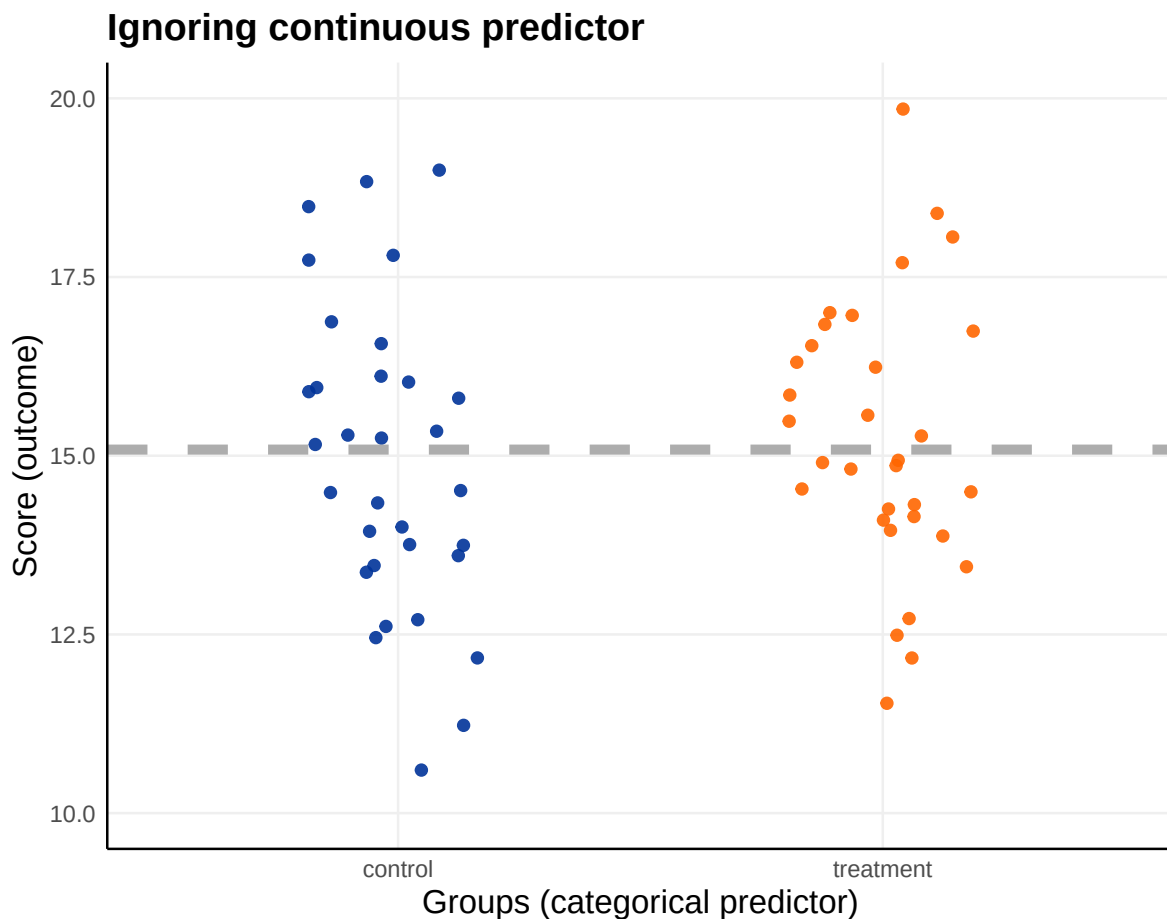


Figure 7: Ignoring a predictor

Predictor x_{1i} on **x-axis** (age), observations on y-axis

Term	Estimate	Std. Error	t value	p value
(Intercept)	14.910	0.356	41.915	3.32×10^{-47}
group ^{treatment}	0.351	0.503	0.698	4.88×10^{-1}

R ²	adj. R ²	σ	statistic	p.value	df	logLik	AIC	BIC	deviance	df.residual	nobs
0.008	-0.008	2.012	0.487	0.488	1.000	-134.550	275.100	281.577	251.061	62.000	64.000

```
lm_int_ign <- lm(score ~ group, data = dta_int)
```

```
tidy_gt(lm_int_ign)
```

There is no group effect !

Remember: conditional on age there is one.

Do not make decisions based on univariable analysis unless you have to.

```
glance_gt(lm_int_ign)
```

Assumptions

Models are based on assumptions,
violations can compromise inference.

For linear regression the assumptions are:

1. **Linearity:** Predictor-outcome relationship is linear
2. **Independence:** Residuals are i.i.d. (no clustering/autocorrelation)
3. **Homoscedasticity:** Constant residual variance
4. **Normality:** Residuals normally distributed (for inference)

Evaluation of model assumption is based on (standardized) residuals.

To evaluate linearity, plot the outcome in relation to

- the fitted value for the whole model
- individual predictors for parts of the model

To evaluate independence, plot the residuals in relation to

- the fitted value for the whole model
- individual predictors for parts of the model

and watch out for patterns

To evaluate homoscedasticity, plot the standardized residuals in relation to

- individual predictors for parts of the model

and watch out for predictor specific spread

```
assumpt <- dta_int |>
  mutate(fitted = fitted(lm_int), resid = residuals(lm_int), resid_std = rstandard(lm_int),
         theoretical = rnorm(length(resid)))

plt_resid_fitted <- ggplot(data = assumpt, aes(x = fitted, y = resid)) + geom_point() +
  geom_hline(yintercept = 0, linetype = "dashed") + # theme_classic() + geom_hline(yintercept = 0, linetype = "dashed") + # theme_classic() + = 0,
  geom_hline(yintercept = 0, linetype = "dashed") + # theme_classic() + linetype = "dashed" + # theme_classic() + = "dashed" + # theme_classic() + + #
  geom_hline(yintercept = 0, linetype = "dashed") + # theme_classic() + theme_classic() + # theme_classic() + + #
  labs(x = "Fitted values", y = "Residuals", title = "Residuals vs fitted")

plt_resid_age <- ggplot(data = assumpt, aes(x = age, y = resid)) + geom_point() +
  geom_hline(yintercept = 0, linetype = "dashed") + # theme_classic() + geom_hline(yintercept = 0, linetype = "dashed") + # theme_classic() + = 0,
  geom_hline(yintercept = 0, linetype = "dashed") + # theme_classic() + linetype = "dashed" + # theme_classic() + = "dashed" + # theme_classic() + + #
  geom_hline(yintercept = 0, linetype = "dashed") + # theme_classic() + theme_classic() + # theme_classic() + + #
  labs(x = "Age", y = "Residuals", title = "Residuals vs age")

plt_resid_group <- ggplot(data = assumpt, aes(x = group, y = resid)) + geom_boxplot() +
  # theme_classic() +
  labs(x = "Group", y = "Residuals", title = "Residuals by group")

wrap_plots(plt_resid_fitted, plt_resid_age, plt_resid_group, nrow = 1)
```

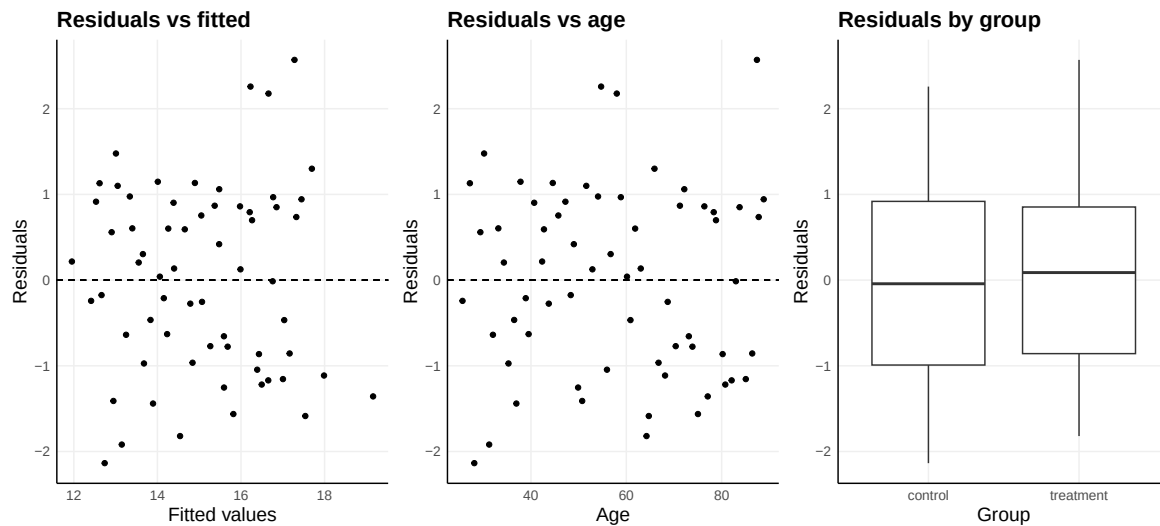


Figure 8: Residual Structure

To evaluate normality of the residuals, use a qqplot.

Note, normality of (standardized) residuals follows from the assumption that disturbances are random and additive.

```
plt_resid_normal <- ggplot(data = assumpt, aes(sample = resid_std)) + stat_qq(distribution =
  color = col1) + stat_qq_line(distribution = qnorm, color = col2) + labs(title = "Q-Q Plot")

plt_resid_hist <- ggplot(data = assumpt) + geom_histogram(aes(x = resid, y = after_stat(dens)),
  bins = 15, alpha = 0.7, fill = col2) + geom_density(aes(x = resid), color = col1,
  linewidth = 1) + labs(title = "Residuals Histogram + Density")

wrap_plots(plt_resid_normal, plt_resid_hist, nrow = 1)
```

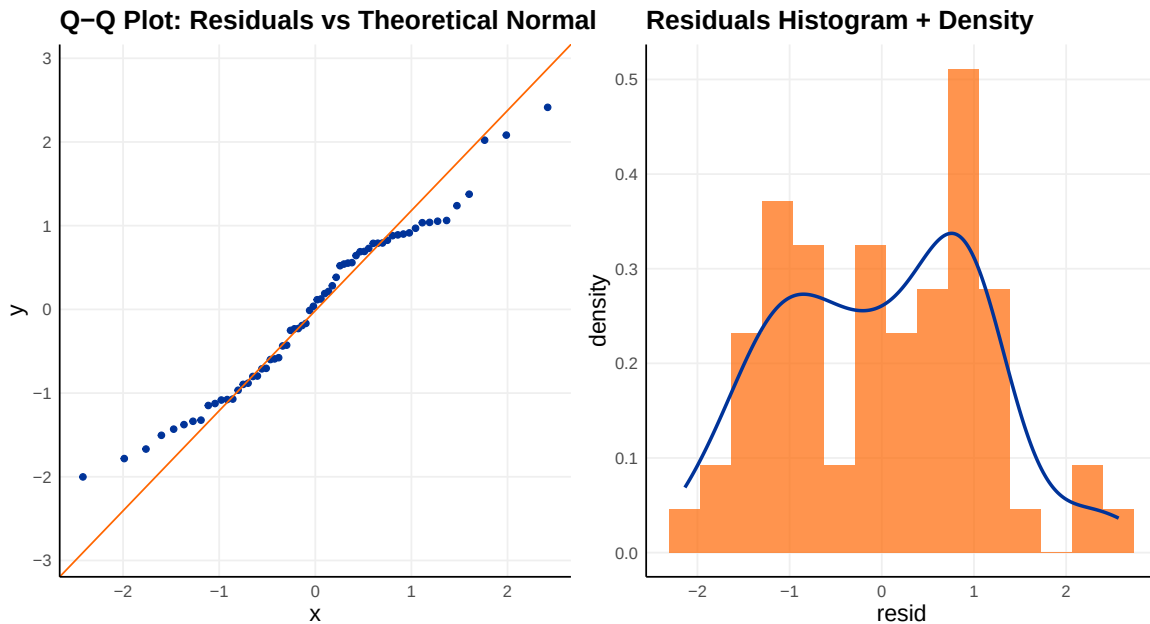


Figure 9: Residual Normality

For the identically and individually distributed assumption:

$$\varepsilon_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2).$$

The lack of any patterns supports the i.i.d. assumption (does not prove it) Remaining patterns in the residuals can possibly be addressed

- using additional variables
- using transformations

Build models to capture the systematicity in the data,
so the remaining part is random, and ideally relatively small.

For completeness,
while tests are just tests:

```
tests <- data.frame(Test = c("Durbin-Watson (Independence)", "Breusch-Pagan (Homoscedasticity)",
  "Shapiro-Wilk (Normality)"), Result = c(round(car::durbinWatsonTest(lm_int)$dw,
  2), round(car::ncvTest(lm_int)$p, 3), round(shapiro.test(residuals(lm_int))$p.value,
  3)), Interpretation = c("2 = no autocorrelation ", "p > 0.05 = constant variance ",
  "p > 0.05 = normality "))
knitr::kable(tests)
```

Table 1: Diagnostics Tests

Test	Result	Interpretation
Durbin-Watson (Independence)	2.030	2 = no autocorrelation
Breusch-Pagan (Homoscedasticity)	0.257	$p > 0.05$ = constant variance
Shapiro-Wilk (Normality)	0.162	$p > 0.05$ = normality

```
save.image("zenzi.RData")
```

Dependent data

Data is often internally dependent:

- cross-sectional clustering: groups of observations (classes, litters, clinics, ...)
- within-unit clustering: repeated measures (trajectories, pre-post, ...)
- distances (less relation when further)
- study effects measuring the same (meta-analysis)
- spatial random effects
- item response models, with respondent clustering
- ...

What dependency looks like:

```
# lm_cont <- lm(score ~ age, data = dta_cont) # use dta_cont, dta_grouped
pltdta_cont <- dta_cont
pltdta_cont$resid <- residuals(lm_cont)
dta_sorted <- pltdta_cont[order(-pltdta_cont$resid), ]
dta_sorted$group <- factor(rep(1:9, times = c(4, 4, 8, 8, 16, 8, 8, 4, 4)))
dta_grouped <- dta_sorted[order(dta_sorted$id), ]
set.seed(123)
dta_grouped$group0 <- factor(sample(c(rep(c(1, 2, 8, 9), 4), rep(c(3, 4, 6, 7), 8),
  rep(5, 16))))
# summary(lme4::lmer(score ~ age + (1 | group), data = dta_grouped))
# summary(lme4::lmer(score ~ age + (1 | group0), data = dta_grouped))
```

```
pltdta_grouped <- dta_grouped
lm_naive_9 <- lm(score ~ age, data = pltdta_grouped)
pltdta_grouped$fitted_naive <- fitted(lm_naive_9)
pltdta_grouped$resid_naive <- residuals(lm_naive_9)
```

```
plt_resid_naive <- ggplot(pltdta_grouped, aes(fitted_naive, resid_naive)) + geom_text(aes(label = group,
  colour = factor(group %in% c(1, 9), levels = c(FALSE, TRUE), labels = c("other",
    "1"))), size = 4) + scale_color_manual(values = c(other = "grey70", `1` = "red"),
  guide = "none") + geom_hline(yintercept = 0, lty = 2, color = "red") + labs(title = "Unaccounted for Groups") +
  ylim(c(-4, 4))
```

```
lm_group_main_9 <- lm(score ~ age + group, data = pltdta_grouped)
pltdta_grouped$fitted_group_main <- fitted(lm_group_main_9)
pltdta_grouped$resid_group_main <- residuals(lm_group_main_9)
```

```
plt_resid_group <- ggplot(pltdta_grouped, aes(fitted_group_main, resid_group_main)) +
  geom_text(aes(label = group, colour = factor(group %in% c(1, 9), levels = c(FALSE,
    TRUE), labels = c("other", "1"))), size = 4) + scale_color_manual(values = c(other =
    `1` = "red"), guide = "none") + geom_hline(yintercept = 0, lty = 2, color = "red") +
  labs(title = "Accounted for Groups") + ylim(c(-4, 4))
```

```
wrap_plots(plt_resid_naive, plt_resid_group, nrow = 1)
```

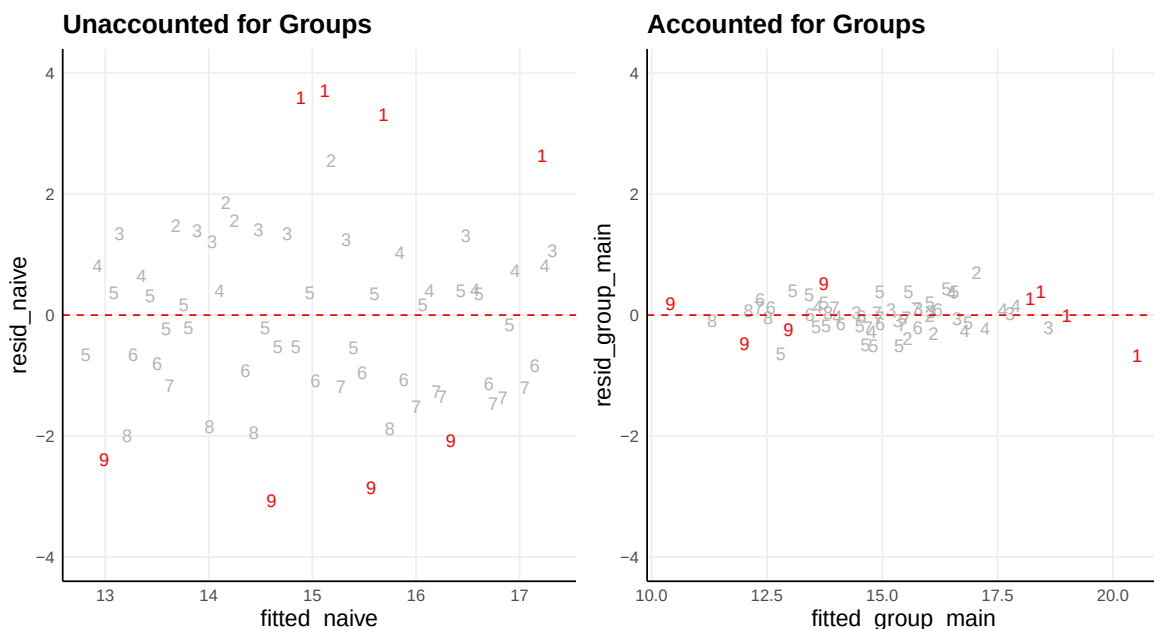


Figure 10: Accounting for Grouping

Note the assumption

$$\varepsilon_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2).$$

All residuals (variance unaccounted for by the model) assumed distributed individually and identically,
all part of the same underlying distribution !

Any residual is fully uninformative about any other residual. All residuals combined reflect one single distribution.

The clustering shows that grouping should be included in the model.

Working Example

Let's assume a SQUARE test exists

- with test results between 0 and 20
- performed on patients
- **part of one of 9 groups of patients**
- who have particular characteristics (e.g., age, ...)
- under particular conditions (e.g., treated or not)

We want to understand the conditions (e.g., treatment) under which test results are higher/lower

- **acknowledging group-members to be more similar to each-other, in average**
- by relating
 - the data (test results ~ outcome) to
 - the conditions (patient properties ~ predictors)
 - → columns
- using information offered by each of the patients (replications ~ sample size)
 - → rows

Regression Model: grouping as Fixed variable

Bringing in groups

- reduces residuals
- results in residuals around 0 for each group
- removed dependence: knowledge of an instance of a group is not informative about the other instances

Independent groups, common age effect

Using a group identification variable to account for groups.

The regression model is the same as before, 9 groups instead of 2:

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 g_i + \varepsilon_i, \quad \text{for } i = 1, \dots, n,$$

with the usual distributional assumption:

$$\varepsilon_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2).$$

```
# use group_colors
lm_group_main_9 <- lm(score ~ age + group, data = dta_grouped)
cont9_stats <- dta_grouped |>
  summarise(mean_score = mean(score), sd_score = sd(score), n = n(), se_mean = sd(score)/sqrt(n),
    .groups = "drop")

pltdta_group_main <- augment(lm_group_main_9, data = dta_grouped) |>
  select(-c(resid)) |>
  rename(fitted = .fitted, resid = .resid)

names(group_colors_9) <- c(1, 5, 9, 2, 6, 3, 7, 4, 8)
pred_data <- expand_grid(age = seq(min(dta_grouped$age), max(dta_grouped$age), length.out = 100),
  group = factor(1:9))
pred_data$fitted <- predict(lm_group_main_9, newdata = pred_data)

pred_data$group_gray <- factor(pred_data$group, levels = unique(pred_data$group)[order(c(1,
  5, 9, 2, 6, 3, 7, 4, 8))])

plt_similarity_high <- ggplot(pltdta_group_main, aes(x = age, y = score)) + geom_point(aes(c
  size = 2.5, alpha = 0.85) + scale_color_manual(values = group_grays_9, guide = "none") +
  geom_segment(data = pltdta_group_main, aes(x = age, xend = age, y = score, yend = fitted),
    color = "gray60", alpha = 0.3, linewidth = 0.8) + geom_line(data = pred_data,
  aes(y = fitted, color = group_gray), linewidth = 1.2, show.legend = FALSE) +
  annotate("text", x = mean(pltdta_group_main$age), y = max(pltdta_group_main$score) +
    0.5, label = paste0("score ", round(coef(lm_group_main_9)[1], 2), " + ",
    round(coef(lm_group_main_9)["age"], 3), " × age + group effect"), size = 4.5) +
  labs(x = "Age (continuous predictor)", y = "Score (outcome)", title = "Equal age effect")
```

```
coord_cartesian(ylim = c(10, 20))
plt_similarity_high
```

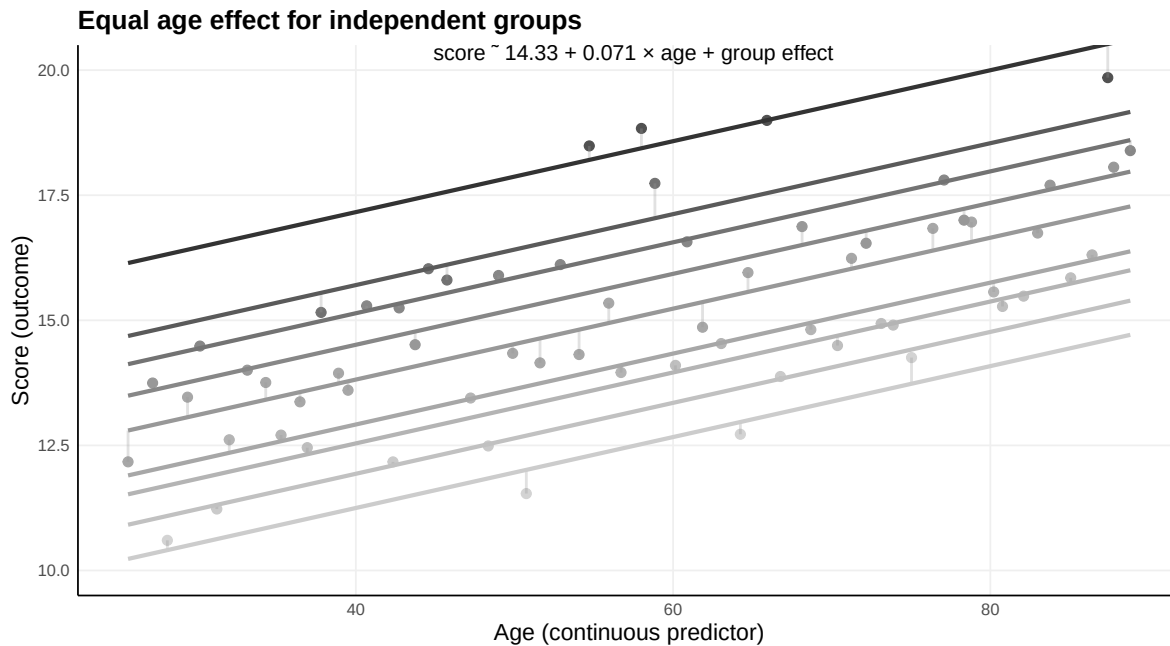


Figure 11: Grouping as Fixed Variable; Age effect considered equal over groups, high within group similarity

Age effect, assumed equal for each of 9 groups.

Age on x-axis, observations on y-axis

- lines represent group specific conditional averages
 - in this case, conditional on age
 - age effect, independent of group

With nine groups in the model.

```
lm_group_main_9 <- lm(score ~ age + group, data = dta_grouped)
```

```
tidy_gt(lm_group_main_9)
```

Term	Estimate	Std. Error	t value	p value
(Intercept)	14.327	0.213	67.238	9.56×10^{-54}
age	0.071	0.002	31.777	1.24×10^{-36}
group2	-1.458	0.221	-6.606	1.80×10^{-8}
group3	-2.021	0.189	-10.696	5.99×10^{-15}
group4	-2.652	0.188	-14.133	8.70×10^{-20}
group5	-3.347	0.174	-19.287	6.97×10^{-26}
group6	-4.245	0.188	-22.564	3.66×10^{-29}
group7	-4.623	0.188	-24.652	4.66×10^{-31}
group8	-5.230	0.221	-23.711	3.20×10^{-30}
group9	-5.914	0.218	-27.133	3.84×10^{-33}

R ²	adj. R ²	σ	statistic	p.value	df	logLik	AIC	BIC	deviance	df.residual	nobs
0.980	0.977	0.306	294.448	0.000	9.000	-9.568	41.136	64.884	5.053	54.000	64.000

```
glance_gt(lm_group_main_9)
```

There is a clear age and group effect, each of the groups differs from the reference group conditional on age.

The Mean Squared Error is huge compared to the Residual Variance.

```
tidy_gt(lm_group_main_9, type = "anova")
```

Group specific age effect

What if we would allow group specific age effects.

The regression model is:

$$y_i = \beta_0 + \beta_1 \cdot x1_i + \beta_2 \cdot g2_i + \beta_3 \cdot x1_i \cdot g2_i + \varepsilon_i, \quad \text{for } i = 1, \dots, n,$$

with the usual distributional assumption:

Term	df	Sum Sq	Mean Sq	F value	p value
age	1.000	111.039	111.039	1,186.612	1.95×10^{-38}
group	8.000	136.942	17.118	182.928	2.80×10^{-36}
Residuals	54.000	5.053	0.094	NA	NA

$$\varepsilon_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2).$$

```
lm_group_int_9 <- lm(score ~ age * group, data = dta_grouped)

pltdta_group_int <- augment(lm_group_int_9, data = select(dta_grouped, -resid)) |>
  rename(fitted = .fitted, resid = .resid)

names(group_colors_9) <- c(1, 5, 9, 2, 6, 3, 7, 4, 8)
pred_data <- expand_grid(age = seq(min(dta_grouped$age), max(dta_grouped$age), length.out = 100),
  group = factor(1:9))
pred_data$fitted <- predict(lm_group_int_9, newdata = pred_data)

pred_data$group_gray <- factor(pred_data$group, levels = unique(pred_data$group)[order(c(1,
  5, 9, 2, 6, 3, 7, 4, 8))])

ggplot(pltdta_group_int, aes(x = age, y = score)) + geom_point(aes(color = group),
  size = 2.5, alpha = 0.85) + scale_color_manual(values = group_grays_9, guide = "none") +
  geom_segment(data = pltdta_group_int, aes(x = age, xend = age, y = score, yend = fitted),
    color = "gray60", alpha = 0.3, linewidth = 0.8) + geom_line(data = pred_data,
    aes(y = fitted, color = group_gray), linewidth = 1.2, show.legend = FALSE) +
  annotate("text", x = mean(pltdta_group_int$age), y = max(pltdta_group_int$score) +
    0.5, label = paste0("score ", round(coef(lm_group_int_9)[1], 2), " + ",
    round(coef(lm_group_int_9)["age"], 3), " × age * group effect"), size = 4.5) +
  labs(x = "Age (continuous predictor)", y = "Score (outcome)", title = "Different age eff
  coord_cartesian(ylim = c(10, 20))
```

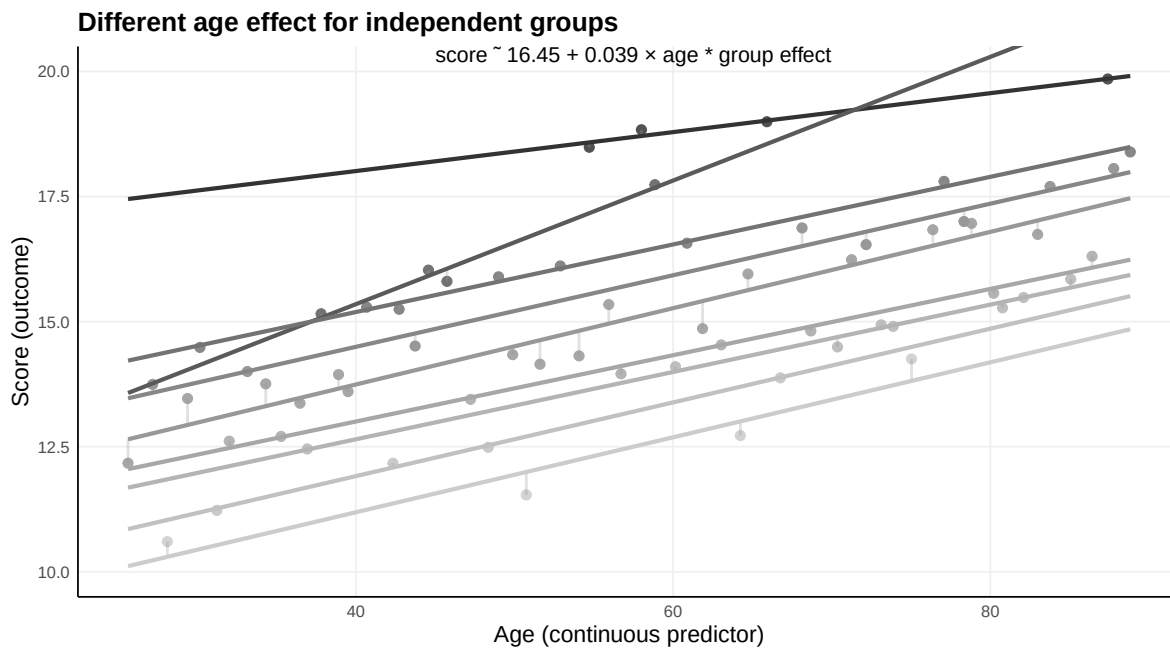


Figure 12: Grouping as Fixed Variable; Age effect allowed to be different over groups, high within group similarity

The age effect is allowed to differ for each of 9 groups.

Age on x-axis, observations on y-axis

- lines represent group specific conditional averages

With nine group in the model.

```
lm_group_int_9 <- lm(score ~ age * group, data = dta_grouped)
```

```
tidy_gt(lm_group_int_9)
```

```
glance_gt(lm_group_int_9)
```

There is a clear age and group effect, each of the groups differs from the reference group conditional on age.

The Mean Squared Error is huge compared to the Residual Variance.

Term	Estimate	Std. Error	t value	p value
(Intercept)	16.453	0.730	22.541	1.69×10^{-26}
age	0.039	0.011	3.611	7.51×10^{-4}
group2	-6.047	1.123	-5.386	2.38×10^{-6}
group3	-3.964	0.793	-5.001	8.75×10^{-6}
group4	-4.818	0.785	-6.141	1.78×10^{-7}
group5	-5.759	0.762	-7.560	1.33×10^{-9}
group6	-6.099	0.798	-7.643	9.98×10^{-10}
group7	-6.496	0.872	-7.447	1.95×10^{-9}
group8	-7.488	0.897	-8.347	9.14×10^{-11}
group9	-8.260	0.857	-9.640	1.29×10^{-12}
age:group2	0.085	0.021	4.033	2.06×10^{-4}
age:group3	0.029	0.012	2.384	2.13×10^{-2}
age:group4	0.033	0.012	2.803	7.39×10^{-3}
age:group5	0.037	0.011	3.259	2.11×10^{-3}
age:group6	0.027	0.012	2.281	2.73×10^{-2}
age:group7	0.028	0.013	2.240	3.00×10^{-2}
age:group8	0.035	0.015	2.293	2.65×10^{-2}
age:group9	0.036	0.013	2.702	9.61×10^{-3}

R ²	adj. R ²	σ	statistic	p.value	df	logLik	AIC	BIC	deviance	df.residual	nobs
0.986	0.981	0.274	194.902	0.000	17.000	2.507	32.986	74.005	3.465	46.000	64.000

Term	df	Sum Sq	Mean Sq	F value	p value
age	1.000	111.039	111.039	1,474.179	1.36×10^{-36}
group	8.000	136.942	17.118	227.259	2.55×10^{-34}
age:group	8.000	1.588	0.199	2.636	1.82×10^{-2}
Residuals	46.000	3.465	0.075	NA	NA

```
tidy_gt(lm_group_int_9, type = "anova")
```

Regression Model: Why this sometimes is not enough

Grouping as fixed variable can be a problem, especially with many small clusters (groups):

- many parameters have to be estimated because many clusters
- for each cluster, parameters poorly estimated when small clusters (overfitting)
- no generalization to clusters not part of the data

```
plt_resid_naive + labs(title = "Clustered Residuals") + ylim(c(-4, 4)) + geom_text(aes(label =
  colour = factor(group %in% c(1, 9), levels = c(FALSE, TRUE), labels = c("other",
    "1"))), size = 4) + scale_color_manual(values = c(other = "grey70", `1` = "red"),
  guide = "none")
| Scale for y is already present.
| Adding another scale for y, which will replace the existing scale.
| Scale for colour is already present.
| Adding another scale for colour, which will replace the existing scale.
```

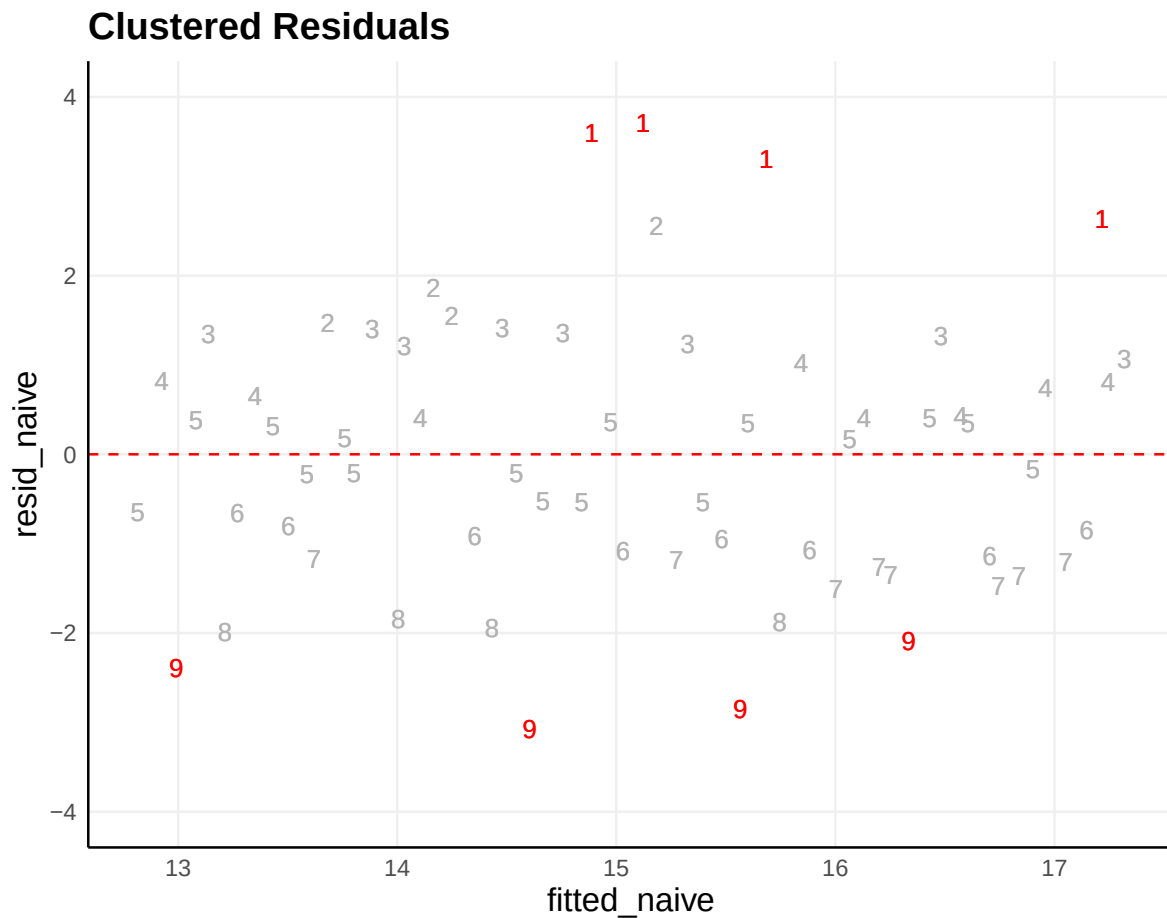


Figure 13: Clustered Residuals

Ignoring grouping is a problem when within group similarities exist (dependencies):

- violation of i.i.d. assumption
- **Standard errors** are too small $\rightarrow$ p-values too small $\rightarrow$ false positives inflate.
- group specific effects ignored too

Solution: model the within group similarities,
once estimated the rest of the model is conditional upon it.

Note: within cluster similarity $\sim$ between cluster difference

Capture dependence through estimating variance.


```
lm_group0_main_9 <- lm(score ~ age + group0, data = dta_grouped)

pltdta_group0_main <- augment(lm_group0_main_9, data = select(dta_grouped, -resid)) |>
  rename(fitted = .fitted, resid = .resid)

names(group_colors_9) <- c(1, 5, 9, 2, 6, 3, 7, 4, 8)
pred_data <- expand_grid(age = seq(min(dta_grouped$age), max(dta_grouped$age), length.out = 100),
  group0 = factor(1:9))
pred_data$fitted <- predict(lm_group0_main_9, newdata = pred_data)

pred_data$group_gray <- factor(pred_data$group0, levels = unique(pred_data$group0)[order(c(1, 5, 9, 2, 6, 3, 7, 4, 8))])

plt_similarity_low <- ggplot(pltdta_group0_main, aes(x = age, y = score)) + geom_point(aes(color = group0,
  size = 2.5, alpha = 0.85)) + scale_color_manual(values = group_grays_9, guide = "none") +
  geom_segment(data = pltdta_group0_main, aes(x = age, xend = age, y = score, yend = fitted),
    color = "gray60", alpha = 0.3, linewidth = 0.8) + geom_line(data = pred_data,
    aes(y = fitted, color = group_gray), linewidth = 1.2, show.legend = FALSE) +
  annotate("text", x = mean(pltdta_group0_main$age), y = max(pltdta_group0_main$score) +
    0.5, label = paste0("score ", round(coef(lm_group0_main_9)[1], 2), " + ",
    round(coef(lm_group0_main_9)["age"], 3), " × age + group effect"), size = 4.5) +
  labs(x = "Age (continuous predictor)", y = "Score (outcome)", title = "Age effect for in") +
  coord_cartesian(ylim = c(10, 20))

plt_similarity_high <- plt_similarity_high + labs(x = "Age (continuous predictor)",
  y = "Score (outcome)", title = "High Similarity")
plt_similarity_low <- plt_similarity_low + labs(x = "Age (continuous predictor)",
  y = "Score (outcome)", title = "Low Similarity")
wrap_plots(plt_similarity_high, plt_similarity_low, nrow = 1)
```

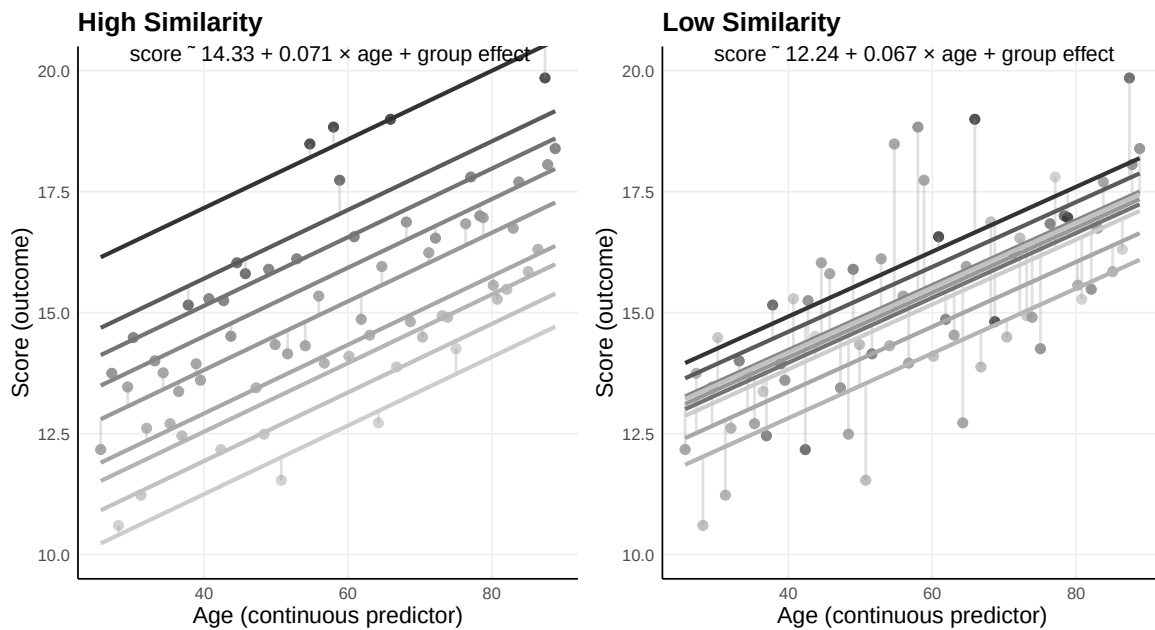


Figure 14: Variance and Correlation, two sides of a coin

```
save.image("zenzi.RData")
```

Mixed Model

Mixed models (a.k.a Multilevel Model)

- extend regression models,
- is modeling variance
- turning it into a conditional mean
- by estimating relations
- by minimizing squared error.
- **while accomodating dependencies**
- **through estimating variances (heterogeneity)**

Random Intercept Model

Let y_{ij} denote the outcome for individual i in cluster j .

Level 1 (case within cluster):

$$y_{ij} = \beta_{0j} + \beta_1 x_{ij} + \varepsilon_{ij}, \quad \varepsilon_{ij} \sim \mathcal{N}(0, \sigma^2)$$

Level 2 (cluster-level model for the intercept):

$$\beta_{0j} = \gamma_{00} + u_{0j}, \quad u_{0j} \sim \mathcal{N}(0, \tau_{00})$$

Combined (reduced form):

$$y_{ij} = \underbrace{\gamma_{00} + \beta_1 x_{ij}}_{\text{fixed part}} + \underbrace{u_{0j} + \varepsilon_{ij}}_{\text{random part}}$$

The random part implies a variance decomposed into random intercept variance and residual variance:

$$\text{Var}(y_{ij}) = \tau_{00} + \sigma^2, \quad \text{Cov}(y_{ij}, y_{i'j}) = \tau_{00} \quad (i \neq i', \text{ same group } j)$$

The relative proportion of heterogeneity for this simple model is the intra class correlation (ICC):

$$\text{ICC} = \tau_{00} / (\tau_{00} + \sigma^2).$$

Why a Random Intercept Model

Mixed models treat cluster effects as **random draws from a distribution**.

Instead of estimating J-1 fixed effects + intercept

$$\beta_0, \beta_1, \dots, \beta_{J-1}$$

random effects are estimated as

$$\gamma_{00} + u_{0j} \quad \text{with} \quad u_{0j} \sim \mathcal{N}(0, \tau_{00})$$

resulting in only 2 parameters to estimate: grand mean γ_{00} and variance τ_{00}

Notice the similarity with the residuals:

$$\varepsilon_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2).$$

A single variance parameter captures cluster heterogeneity

With a growing number of clusters

- no increase in number of parameters to estimate
- a better estimate of the variance parameter

In this particular case:

```
lmer_int_9 <- lmer(score ~ age + (1 | group), data = dta_grouped)

cont9_stats <- dta_grouped |>
  summarise(mean_score = mean(score), sd_score = sd(score), n = n(), se_mean = sd(score)/sqrt(n),
    .groups = "drop")

pltdta_ri_9_high <- augment(lmer_int_9, data = dta_grouped) |>
  select(-resid) |>
  rename(fitted = .fitted, resid = .resid)

pred_data <- expand_grid(age = seq(min(dta_grouped$age), max(dta_grouped$age), length.out = 100),
  group = factor(1:9))

pred_data$fitted <- predict(lmer_int_9, newdata = pred_data, re.form = NULL)
pred_data$group_gray <- factor(pred_data$group, levels = unique(pred_data$group)[order(c(1,
  5, 9, 2, 6, 3, 7, 4, 8))])

plt_mixed_similarity_high <- ggplot(pltdta_ri_9_high, aes(x = age, y = score)) +
  geom_point(aes(color = factor(group)), size = 2.5, alpha = 0.85) + scale_color_manual(values = c("red", "blue", "green", "orange", "purple", "brown", "pink", "gray", "black")) +
  guide = "none") + geom_segment(data = pltdta_ri_9_high, aes(x = age, xend = age,
  y = score, yend = fitted), color = "gray60", alpha = 0.3, linewidth = 0.8) +
  geom_line(data = pred_data, aes(y = fitted, color = group_gray), linewidth = 1.2,
    show.legend = FALSE) + annotate("text", x = mean(pltdta_ri_9_high$age), y = max(pltdta_ri_9_high$score) * 0.5, label = paste0("score ", round(fixef(lmer_int_9)[1], 2), " + ", round(fixef(lmer_int_9)[2], 2), " × age + random effect"), size = 4.5) + labs(x = "Age (continuous predictor)",
  y = "Score (outcome)", title = "Mixed Model") + coord_cartesian(ylim = c(10, 20))

plt_fixed_similarity_high <- plt_mixed_similarity_high + labs(x = "Age (continuous predictor)",
  y = "Score (outcome)", title = "Fixed model")

wrap_plots(plt_mixed_similarity_high, plt_fixed_similarity_high, nrow = 1)
```

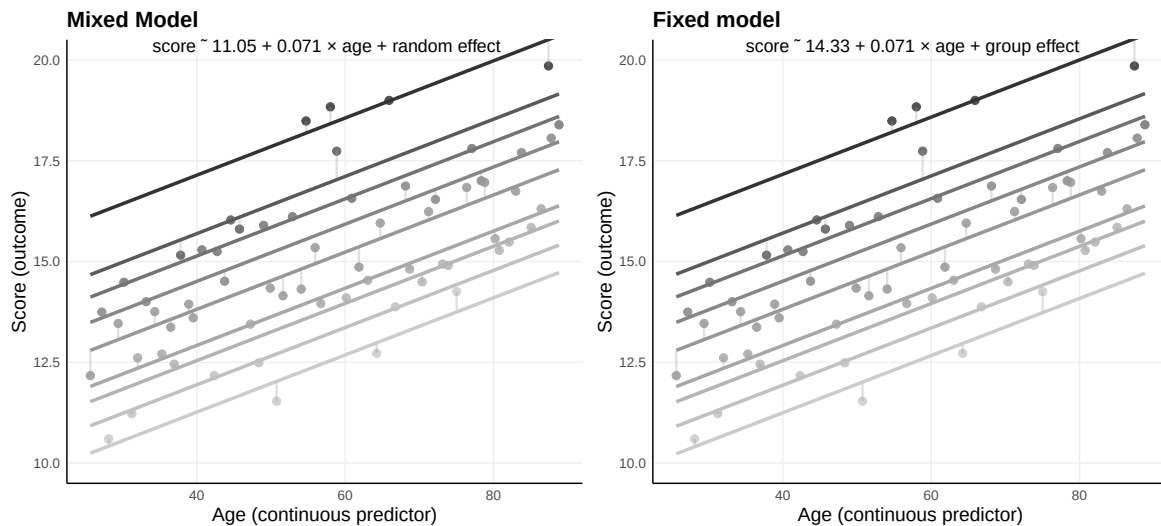


Figure 15: Random vs. Fixed: in this particular case almost identical

```
# maybe change later to tidy_gt and tidy_mixed_gt
t_lm <- tidy(lm_group_main_9) |>
  select(term, estimate, std.error, statistic) |>
  mutate(across(c(estimate, std.error, statistic), ~round(.x, 3)))
m_lm <- glance(lm_group_main_9) |>
  select(sigma) |>
  mutate(sigma = round(sigma, 3))
t_lmer <- tidy(lmer_int_9) |>
  select(term, estimate, std.error, df, statistic) |>
  mutate(across(c(estimate, std.error, statistic, df), ~round(.x, 3)))
```

Mixed

in R:

```
lmer(score ~ age + (1 | group),
data = ..)
```

```
gt(t_lmer)
```

Cluster specific estimates extracted:

```
gt(ranef(lmer_int_9)$group)
```

term	estimate	std.error	df	statistic
(Intercept)	11.049	0.652	8.621	16.943
age	0.071	0.002	54.054	31.797
sd__(Intercept)	1.914	NA	NA	NA
sd__Observation	0.306	NA	NA	NA

(Intercept)
3.25578815
1.80731994
1.25143614
0.62302063
-0.07019019
-0.96517266
-1.34190800
-1.94038853
-2.61990547

```
vc <- as.data.frame(VarCorr(lmer_int_9))
tau00_hat <- vc$vcov[vc$grp == "group"]
sigma2_hat <- vc$vcov[vc$grp == "Residual"]
icc_val <- tau00_hat/(tau00_hat + sigma2_hat)
icc_vals <- c(tau_00 = tau00_hat, sigma2 = sigma2_hat, ICC = icc_val)
icc_df <- data.frame(icc_vals) %>%
  rownames_to_column("metric")
gt(icc_df)
```

Fixed
in R
lm(score ~ age + group,
data = ..)

metric	icc_vals
tau_00	3.6648637
sigma2	0.0935955
ICC	0.9750974

term	estimate	std.error	statistic
(Intercept)	14.327	0.213	67.238
age	0.071	0.002	31.777
group2	-1.458	0.221	-6.606
group3	-2.021	0.189	-10.696
group4	-2.652	0.188	-14.133
group5	-3.347	0.174	-19.287
group6	-4.245	0.188	-22.564
group7	-4.623	0.188	-24.652
group8	-5.230	0.221	-23.711
group9	-5.914	0.218	-27.133
sigma			
0.306			

gt(t_lm)

Fixed, residuals

gt(m_lm)

Level specific slopes

Different levels can have different predictors of the observations (at level 1).

- example: test results explained by patient (level 1) hospital (level 2) variables
 - e.g., older patients (age) result in higher scores
 - e.g., university hospitals (type) result in higher scores

Note: Variables can only explain variability at the lower or equal level.

Let y_{ij} denote the outcome for individual i in cluster j .

Level 1 (case within cluster):

$$y_{ij} = \beta_{0j} + \beta_1 x_{ij} + \varepsilon_{ij}, \quad \varepsilon_{ij} \sim \mathcal{N}(0, \sigma^2)$$

Level 2 (cluster-level model for the intercept, with a Level-2 predictor z_j):

$$\beta_{0j} = \gamma_{00} + \gamma_{01} z_j + u_{0j}, \quad u_{0j} \sim \mathcal{N}(0, \tau_{00})$$

Combined (reduced form):

$$y_{ij} = \underbrace{\gamma_{00} + \beta_1 x_{ij} + \gamma_{01} z_j}_{\text{fixed part}} + \underbrace{u_{0j} + \varepsilon_{ij}}_{\text{random part}}$$

The same variable can sometimes be split up and defined at different levels. Let y_{ij} denote the outcome for individual i in cluster j .

Level 1 (case within cluster):

$$y_{ij} = \beta_{0j} + \beta_{1W} x_{ijW} + \varepsilon_{ij}, \quad \varepsilon_{ij} \sim \mathcal{N}(0, \sigma^2)$$

where

$x_{ijW} = x_{ij} - \bar{x}_j$ (within-cluster centered x)

$\bar{x}_j = \frac{1}{n_j} \sum_{i=1}^{n_j} x_{ij}$ (cluster mean of x)

Level 2 (cluster-level model for the intercept, now including the cluster mean):

$$\beta_{0j} = \gamma_{00} + \gamma_{01} \bar{x}_j + u_{0j}, \quad u_{0j} \sim \mathcal{N}(0, \tau_{00})$$

Combined (reduced form):

$$y_{ij} = \underbrace{\gamma_{00} + \beta_{1W} x_{ijW} + \gamma_{01} \bar{x}_j}_{\text{fixed part}} + \underbrace{u_{0j} + \varepsilon_{ij}}_{\text{random part}}$$

A fixed slope, maybe age, or weight, ... split in

- within cluster centered slope
- cluster specific average

One part focusses on within cluster relative position, other on cluster properties.

Example: blood pressure explained by body weight

- patients with relatively higher weights in a hospital may have higher blood pressure
- hospitals may differ in average weight because of ...

Without this split, it could be a blend.

Note: ecological fallacy.

```
dta_grouped_center <- dta_grouped |>
  group_by(group) |>
  mutate(agem = mean(age), agec = age - agem)
```



```
# Plot 1: Within-group part (Level 1) - agec (centered age)
plt_center_within <- ggplot(data = dta_grouped_center, aes(x = agec, y = score)) +
  geom_point(alpha = 0.3) + geom_line(aes(group = group), alpha = 0.5) + geom_smooth(aes(g
method = "lm", se = FALSE, color = "blue", linewidth = 1) + labs(title = "Within-group (L
x = "Age (centered within group)", y = "Score") + theme_minimal()

# Plot 2: Between-group part (Level 2) - group means
dta_group_means <- dta_grouped_center |>
  group_by(group) |>
  summarise(score_mean = mean(score), agem = mean(age), .groups = "drop") |>
  mutate(group = factor(group), group_ordered = factor(group, levels = order(agem))) # or

plt_center_between <- ggplot(data = dta_group_means, aes(x = group_ordered, y = score_mean))
  geom_point(size = 3) + geom_line(aes(group = 1), alpha = 0.5) + geom_smooth(method = "lm
se = FALSE, color = "red", linewidth = 1) + labs(title = "Between-group (Level 2)",
x = "Group (ordered by mean age)", y = "Mean Score") + theme_minimal() + theme(axis.text
hjust = 1))

# Combine plots
plt_center_within + plt_center_between + plot_layout(nrow = 1)
| `geom_smooth()` using formula = 'y ~ x'
| `geom_smooth()` using formula = 'y ~ x'
```

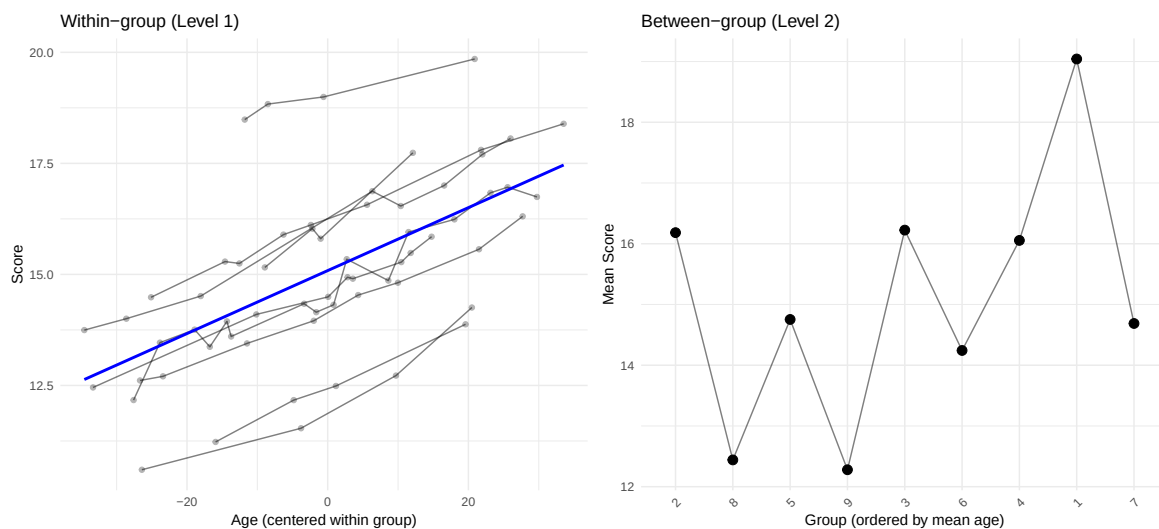


Figure 16: Within and Between Group Differences

effect	group	term	estimate	std.error	statistic	df	p.value
fixed	NA	(Intercept)	11.0488	0.6521	16.9431	8.6207	0
fixed	NA	age	0.0709	0.0022	31.7967	54.0537	0
ran_pars	group	sd__(Intercept)	1.9144	NA	NA	NA	NA
ran_pars	Residual	sd__Observation	0.3059	NA	NA	NA	NA

nobs	sigma	logLik	AIC	BIC	REMLcrit	df.residual
64	0.306	-43.568	95.136	103.771	87.136	60

```
lmer_int_global_9 <- lmer(score ~ age + (1 | group), data = dta_grouped_center)
lmer_int_center_9 <- lmer(score ~ agem + agec + (1 | group), data = dta_grouped_center)
```

Repeating just fitting a slope.

```
tidy_mixed_gt(lmer_int_global_9)
```

```
glance_mixed_gt(lmer_int_global_9)
```

Splitting up over levels.

```
tidy_mixed_gt(lmer_int_center_9)
```

```
glance_mixed_gt(lmer_int_center_9)
```

For the data, a general slope can be fitted, and alternatively, a within and a between slope.

```
lmer_int_slope_0_18 <- lmer(score ~ age + (1 | group), data = dta_grouped_center)
lmer_int_slope_C_18 <- lmer(score ~ agem + agec + (1 | group), data = dta_grouped_center)
```

effect	group	term	estimate	std.error	statistic	df	p.value
fixed	NA	(Intercept)	8.8772	5.1324	1.7296	6.9933	0.1274
fixed	NA	agem	0.1089	0.0890	1.2233	6.9917	0.2609
fixed	NA	agec	0.0709	0.0022	31.7744	53.9835	0.0000
ran_pars	group	sd__(Intercept)	2.0212	NA	NA	NA	NA
ran_pars	Residual	sd__Observation	0.3059	NA	NA	NA	NA

nobs	sigma	logLik	AIC	BIC	REMLcrit	df.residual
64	0.306	-44.999	99.997	110.792	89.997	59

```
dta_grouped_x2 <- dta_grouped_center
lmer_int_slope_0_18 <- lmer(score ~ age + (1 | group), data = dta_grouped_x2)
lmer_int_slope_C_18 <- lmer(score ~ age_ind + age_group + (1 | group), data = dta_grouped_x2)
  group_by(group) |>
  mutate(age_group = mean(age)) |>
  mutate(age_ind = age - age_group))
```

```
tidy_mixed_gt(lmer_int_slope_0_18)
glance_mixed_gt(lmer_int_slope_0_18)
```

```
tidy_mixed_gt(lmer_int_slope_C_18)
glance_mixed_gt(lmer_int_slope_C_18)
```

Relative age matters within clusters,
Cluster average ages does not matter.

```
save.image("zenzi.RData")
```

Borrowing Strength: Partial Pooling

Strategies for estimating each clusters' intercept:

1. **No pooling**: independent clusters, ignores information from other clusters
2. **Complete pooling**: grand mean, ignores group membership
3. **Partial pooling**: reconciles both, compromise

Partial pooling, the key mixed model characteristic

- cluster specific estimates
- more global estimates

Possible because clusters sampled from an underlying population

- other clusters are informative
- use dependency through common population of clusters

Partial pooling implies shrinkage:

shrink each group estimate toward the grand mean

results in random effects $\hat{u}_{0j}$ or **Best Linear Unbiased Predictor (BLUP)**:

$$\hat{u}_{0j} = \underbrace{\frac{n_j/\sigma^2}{n_j/\sigma^2 + 1/\tau_{00}}}_{\text{reliability weight } \lambda_j} \cdot (\bar{y}_{.j} - \hat{\gamma}_{00})$$

The weight defines the percentage with which cluster specific deviation from the global average shrink

- $\lambda_j \rightarrow 1$ with increasing cluster sizes and stronger within cluster correlations (trust the raw data)
- $\lambda_j \rightarrow 0$ with decreasing cluster sizes and weaker within cluster correlations (shrink toward the overall mean).

```
set.seed(123)
# combined 7, 8, 9, to get 16 at the bottom split 6 in sizes 4, 4, and 8
# randomly assign in 1, 2, 3 and 4

dta_grouped_arrange_9 <- dta_grouped
dtb1 <- dta_grouped_arrange_9 |>
  filter(group %in% c(1:4)) |>
  mutate(group1 = sample(group))
dtb2 <- dta_grouped_arrange_9 |>
  filter(group %in% c(5)) |>
  mutate(group1 = factor(sample(c(rep(5, 4), rep(6, 4), rep(8, 8)))))
dtb3 <- dta_grouped_arrange_9 |>
  filter(group %in% c(7)) |>
  mutate(group1 = factor(7))
dtb4 <- dta_grouped_arrange_9 |>
  filter(group %in% c(7, 8, 9)) |>
  mutate(group1 = factor(9))
dta_grouped_arrange_9 <- bind_rows(dtb1, dtb2, dtb3, dtb4)

lmer_int_9x <- lmer(score ~ age + (1 | group1), data = dta_grouped_arrange_9)
grand_mu <- fixef(lmer_int_9x)[["(Intercept)"]]

raw_means <- do.call(rbind, lapply(split(dta_grouped_arrange_9 |>
  group_by(group1) |>
```

```
mutate(n_obs = n()), dta_grouped_arrange_9$group1), function(d) {
  data.frame(group = as.character(d$group1[1]), n_obs = d$n_obs[1], raw_mean = mean(d$score),
    fixef(lmer_int_9x)[[2]] * mean(d$age))
}))

re_vals <- ranef(lmer_int_9x)$group
re_df <- data.frame(group = rownames(re_vals), u0 = re_vals[, 1])
re_df$blup <- grand_mu + re_df$u0

pp_data <- merge(raw_means, re_df, by = "group")
pp_data$gord <- factor(pp_data$group, levels = as.character(1:nrow(pp_data)))

ggplot(pp_data, aes(x = gord)) + geom_hline(yintercept = grand_mu, linetype = "dashed",
  color = "grey50", linewidth = 1) + geom_segment(aes(xend = gord, y = raw_mean,
  yend = blup), arrow = arrow(length = unit(0.18, "cm"), type = "closed"), color = "#e05c2a",
  linewidth = 0.7) + geom_point(aes(y = raw_mean, size = n_obs), color = "#2c6fad",
  alpha = 0.85) + geom_point(aes(y = blup), shape = 18, size = 3.5, color = "#3aab6d") +
  scale_size_continuous(range = c(2, 8), breaks = c(3, 8, 15, 30, 50)) + annotate("text",
  x = 12, y = grand_mu + 1.5, label = "Grand mean", size = 3, color = "grey50",
  hjust = 1) + labs(title = "Partial Pooling: Shrinkage toward the Grand Mean",
  subtitle = "Blue = no-pooling estimate | Green = LMM BLUP | Arrow = shrinkage",
  x = "Group", y = "Estimated intercept", size = "Group n") + theme(legend.position = "right")
```

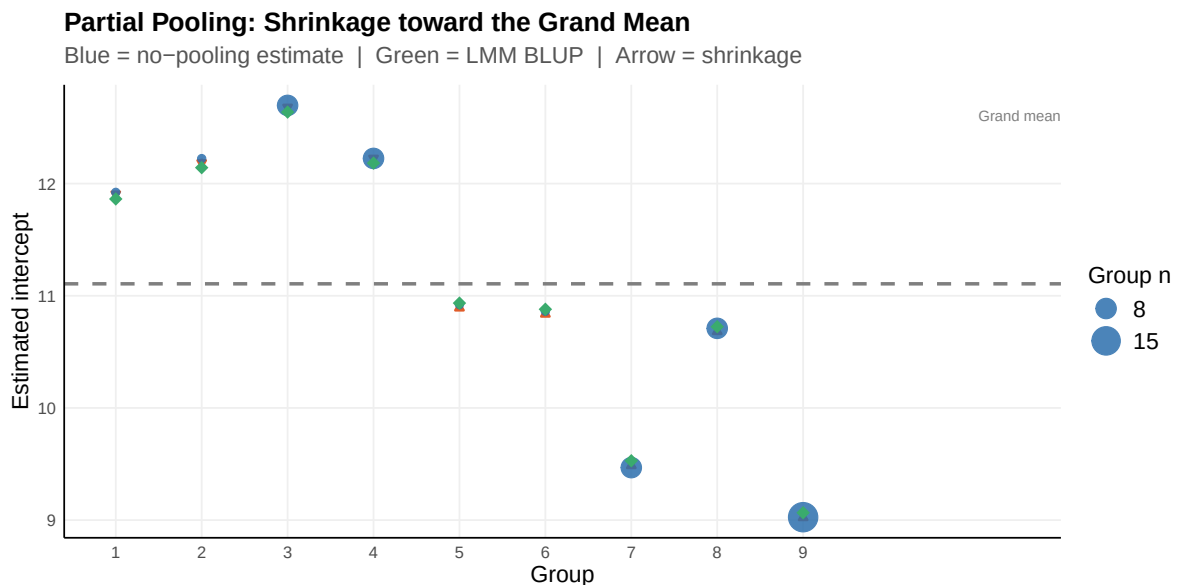


Figure 17: Partial pooling. blue = no-pooling, green = LMM BLUPs

Note: modeling the correlation to shrink estimates allows for cluster specific effects not explicitly represented

Example: matching is not a reason to turn to mixed models, matching uses explicitly available information that should be included in the model as such

Within-Cluster Correlation and Effective Sample Size

Consider two types of predictors:

1. **Between-cluster predictor** Z_j (e.g. hospital type)
 - Only varies between clusters, constant within a cluster.
2. **Within-cluster predictor** x_{ij} (e.g. treatment dose)
 - Varies within clusters.

For a balanced design with J clusters and m observations per cluster:

- Variance of the estimator for the **between-cluster** effect is approximately proportional to

$$\text{Var}(\hat{\gamma}_{01}) \propto \frac{\tau_{00} + \sigma^2/m}{J}.$$

- Variance of the estimator for the **within-cluster** effect is approximately proportional to

$$\text{Var}(\hat{\gamma}_{10}) \propto \frac{\sigma^2}{J \cdot m}.$$

As ρ increases (i.e. τ_{00} grows relative to σ^2):

- $\tau_{00} + \sigma^2/m$ becomes larger $\Rightarrow$ **between-cluster** effects become harder to detect.
- σ^2 becomes smaller relative to the total variance $\Rightarrow$ **within-cluster** effects become easier to detect.

Stronger within-cluster correlation makes

- **between-cluster differences** harder to detect.
- **within-cluster differences** easier to detect.

Note: cluster randomization focuses on between cluster effects, dependent more on the number of clusters, less on the number of observations.

To compare treatments,

randomizing within each cluster is usually far more efficient

```
save.image("zenzi.RData")
```

Adding a Random Slope

Heterogeneity of Effects

A cluster specific slope (effect) can be considered sampled from a common population, with a mean and variance. This is equivalent to cluster specific intercepts.

Level 1:

$$y_{ij} = \beta_{0j} + \beta_{1j}x_{ij} + \varepsilon_{ij}, \quad \varepsilon_{ij} \sim \mathcal{N}(0, \sigma^2)$$

Level 2:

$$\beta_{0j} = \gamma_{00} + u_{0j}, \quad \beta_{1j} = \gamma_{10} + u_{1j}$$

Distributional assumption for the random effects:

$$\begin{pmatrix} u_{0j} \\ u_{1j} \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \Psi\right), \quad \Psi = \begin{pmatrix} \tau_{00} & \tau_{01} \\ \tau_{01} & \tau_{11} \end{pmatrix}$$

Combined:

$$y_{ij} = \underbrace{\gamma_{00} + \gamma_{10}x_{ij}}_{\text{fixed}} + \underbrace{u_{0j} + u_{1j}x_{ij} + \varepsilon_{ij}}_{\text{random}}$$

Ψ captures:

- τ_{00} : how much cluster intercepts vary
- τ_{11} : how much cluster slopes vary
- τ_{01} : covariance between intercept and slope (e.g., high-baseline clusters may also have steeper slopes)

```
dta2 <- dta_grouped |>
  group_by(group) |>
  mutate(av = mean(score)) |>
  ungroup() |>
  mutate(av = av - mean(score), agec = age - mean(age)) |>
  mutate(score = score + (av * 0.05 * agec))
dta2x <- bind_rows(dta_grouped, dta2 |>
  select(-c(av, agec)) |>
  mutate(group = group + 10))
lmer(score ~ age + (1 + age | group), data = dta2x)
ggplot(data = dta2x, aes(y = score, x = age, group = group)) + geom_line() + geom_point()
```

```
dta_grouped_x2 <- dta_grouped |>
  group_by(group) |>
  mutate(av = mean(score)) |>
  ungroup() |>
  mutate(av = av - mean(score), agec = age - mean(age)) |>
  mutate(score = score + (av * 0.05 * agec))
dta_grouped_x2 <- bind_rows(dta_grouped, dta_grouped_x2 |>
  select(-c(av, agec)) |>
  mutate(group = factor(as.numeric(as.character(group)) + 10)))
# ggplot(data=dta_grouped_x2,aes(y=score,x=age,group=group)) + geom_line() +
# geom_point()
```

```
ggplot(dta_grouped_x2, aes(x = age, y = score, color = factor(group))) + geom_point(alpha = 0.5,
  size = 1.5) + geom_smooth(method = "lm", se = FALSE) + geom_smooth(aes(group = 1),
  method = "lm", color = "black", linetype = 2) + scale_color_manual(values = gray.colors(10),
  start = 0.2, end = 0.8), guide = "none") + labs(title = "Increased variability of slopes",
  x = "age", y = "score", color = "Group")
| `geom_smooth()` using formula = 'y ~ x'
| `geom_smooth()` using formula = 'y ~ x'
```


effect	group	term	estimate	std.error	statistic	df	p.value
fixed	NA	(Intercept)	10.9977	0.7236	15.1978	16.5498	0e+00
fixed	NA	age	0.0736	0.0166	4.4196	16.7056	4e-04
ran_pars	group	sd__(Intercept)	3.0292	NA	NA	NA	NA
ran_pars	group	cor__(Intercept).age	-0.8820	NA	NA	NA	NA
ran_pars	group	sd__age	0.0701	NA	NA	NA	NA
ran_pars	Residual	sd__Observation	0.2751	NA	NA	NA	NA

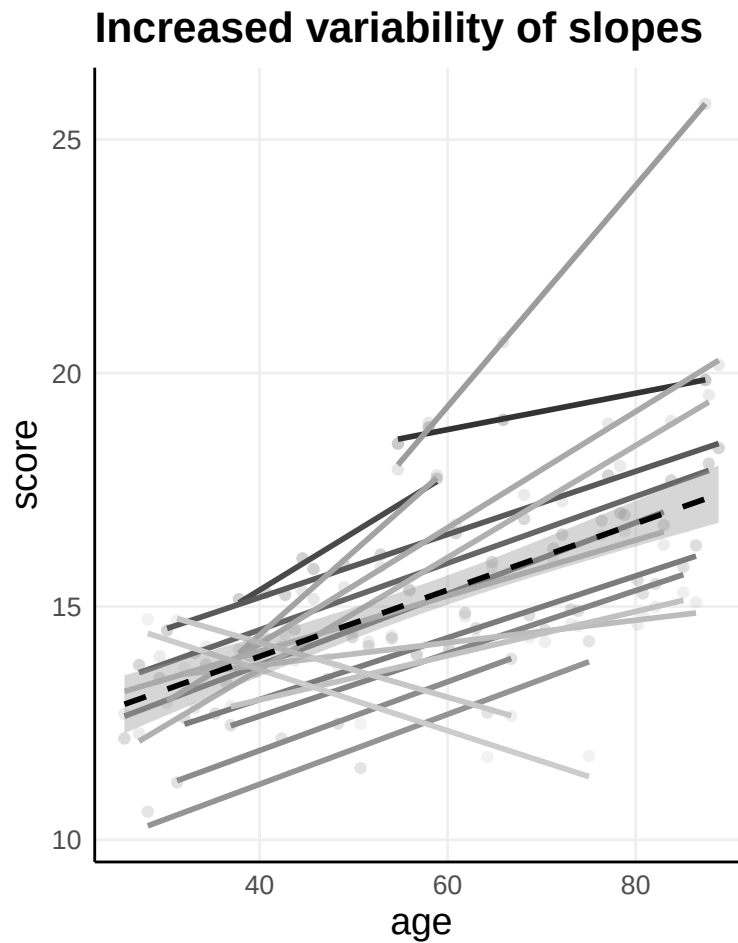


Figure 18: Random intercept + random slope model.

```
lmer_slope_18 <- lmer(score ~ age + (1 + age | group), data = dta_grouped_x2)
tidy_mixed_gt(lmer_slope_18)
```

nobs	sigma	logLik	AIC	BIC	REMLcrit	df.residual
128	0.275	-106.28	224.561	241.673	212.561	122

effect	group	term	estimate	std.error	statistic	df	p.value
fixed	NA	(Intercept)	11.0488	0.6522	16.9413	8.6176	0
fixed	NA	age	0.0709	0.0022	31.7972	54.0546	0
ran_pars	group	sd__(Intercept)	1.9146	NA	NA	NA	NA
ran_pars	group.1	sd__age	0.0000	NA	NA	NA	NA
ran_pars	Residual	sd__Observation	0.3059	NA	NA	NA	NA

```
glance_mixed_gt(lmer_slope_18)
```

Convergence problems:

- few groups (< 10–20), covariance Ψ can be hard to estimate reliably
 - center the predictors to remove correlation random intercept - random slope
 - avoid estimation of covariance: (1 + age || group) which constrains $\tau_{01} = 0$.

```
tidy_mixed_gt(lmer(score ~ age + (1 + age || group), data = dta_grouped))
| boundary (singular) fit: see help('isSingular')
```

Technicalities

Statistical inference is about

- statistical estimation → size
 - estimate: my best guess
 - standard error: uncertainty about my best guess
- statistical testing → decision
 - p-value: probability that by chance
 - change in AIC: improved model

ML vs REML in Mixed Models

Estimation typically by **Maximum Likelihood (ML)** or **Restricted Maximum Likelihood (REML)**.

Optimize likelihood function

- ML estimates **fixed effects and variance components together**
- REML estimate **the variance components** on the information after removal of fixed effects

REML reduces small-sample bias in variance estimates (small number of clusters)

Use **REML** when

- estimation of **variance components** / **random effects**.
- fitting a *final* model

Use **ML** when

- comparison of **nested models with different fixed-effects structure**
- focus on testing or selecting fixed effects

Testing Variance Components

For variance components, the null hypothesis typically assumes variances at the **boundary** of the parameter space (e.g. $\tau^2 = 0$).

The asymptotic distribution of the **likelihood ratio test (LRT)** can be considered a **mixture of chi-squares**.

Example: testing a random intercept (single variance)

Null hypothesis: no between-cluster variability / heterogeneity

Test $H_0 : \tau_{00} = 0$ vs $H_1 : \tau_{00} > 0$.

Fit both models by **ML** (not REML)

```
# H1: random intercept model
mod1_ml <- lmer(score ~ age + (1 | group), data = dta_grouped, REML = FALSE)

# H0: no random intercept (ordinary regression with group-ignored random part)
mod0_ml <- gls(score ~ age, data = dta_grouped)

# Put on a comparable ML deviance scale
logLik_mod1 <- logLik(mod1_ml)
logLik_mod0 <- logLik(mod0_ml)

LR_stat <- 2 * (as.numeric(logLik_mod1) - as.numeric(logLik_mod0))
```

The Likelihood Ratio test statistics is computed

```
LR_stat <- 2 * (as.numeric(logLik_mod1) - as.numeric(logLik_mod0))
```

Under H_0 , the variance component is on the boundary ($\tau_{00} = 0$), the LRT does **not** follow χ_1^2 , but rather a **mixture** of chi-square distributions:

$$\frac{1}{2}\chi_0^2 + \frac{1}{2}\chi_1^2$$

where χ_0^2 is a point mass at 0.

This leads to a simple p-value adjustment: take half !

Evaluate the Likelihood ratio test statistics:

$$p = 0.5 \cdot P(\chi_1^2 \geq \text{LR})$$

```
# mixture 0.5 * chi^2_0 + 0.5 * chi^2_1
p_mix <- 0.5 * (1 - pchisq(LR_stat, df = 1))
p_naive <- 1 - pchisq(LR_stat, df = 1)

c(LR = LR_stat, p_mixture = p_mix, p_naive_chisq1 = p_naive)
```

Beyond a single variance

Testing more complex hypothesis,
about multiple variances and/or covariances

- e.g., testing a **random slope variance**
- e.g., testing a **variance and a covariance simultaneously**

asymptotic LRT distribution remains **finite mixture of chi-squared distributions**, with **degrees of freedom** and **weights** in the mixture depend on

- number of constrained variance and covariance parameters
- structure of the covariance matrix H_0 and H_1

More complex models require simulation.

Note: testing covariance is not a challenge

```
save.image("zenzi.RData")
```

Additional Nesting

Nesting Within Nesting

Data can be nested at more than two levels: patients within wards within hospitals.

Let y_{ijk} = outcome for patient i , in ward j , in hospital k .

Level 1 (patient within ward):

$$y_{ijk} = \pi_{0jk} + \pi_1 x_{ijk} + e_{ijk}, \quad e_{ijk} \sim \mathcal{N}(0, \sigma^2)$$

Level 2 (ward within hospital):

$$\pi_{0jk} = \underline{\beta_{00k}} + r_{0jk}, \quad r_{0jk} \sim \mathcal{N}(0, \tau_\pi^2)$$

Level 3 (hospital):

$$\underline{\beta_{00k}} = \gamma_{000} + u_{00k}, \quad u_{00k} \sim \mathcal{N}(0, \tau_\beta^2)$$

Combined:

$$y_{ijk} = \underbrace{\gamma_{000} + \pi_1 x_{ijk}}_{\text{fixed}} + \underbrace{u_{00k} + r_{0jk} + e_{ijk}}_{\text{three random terms}}$$

Additional **variance components** are included to capture heterogeneity:

- τ_β^2 (between hospitals)
- τ_π^2 (between wards within hospitals)
- σ^2 (within wards).

Note: different levels are assumed independent,
within a level covariance can exist,
between levels covariance is not possible.

```
set.seed(21)
n_schools3 <- 4
n_classes <- 3
n_students3 <- 20

school_u <- rnorm(n_schools3, 0, 8)
class_r <- rnorm(n_schools3 * n_classes, 0, 4)

df_3lvl <- do.call(rbind, lapply(1:n_schools3, function(s) {
  do.call(rbind, lapply(1:n_classes, function(cl) {
    x <- runif(n_students3, 0, 10)
    y <- 50 + school_u[s] + class_r[(s - 1) * n_classes + cl] + 1.5 * x + rnorm(n_students3, 0, 3)
    data.frame(x = x, y = y, school = factor(s, labels = paste("School", s)),
              class = factor((s - 1) * n_classes + cl))
  })))
}))

ggplot(df_3lvl, aes(x, y, color = class)) + geom_point(alpha = 0.5, size = 1.8) +
  geom_smooth(method = "lm", se = FALSE, linewidth = 0.9, aes(group = class)) +
  facet_wrap(~school, nrow = 1) + scale_color_manual(values = rep(pal[c(1, 3, 5)],
4)) + labs(title = "Three-Level Structure: Students within Classes within Schools",
  subtitle = "Each panel = one school; colours = classes within that school", x = "SES",
  y = "Test score", color = "Class") + guides(color = "none")

m_3lvl <- lmer(y ~ x + (1 | school/class), data = df_3lvl)

vc3 <- as.data.frame(VarCorr(m_3lvl))
vc3$pct <- round(100 * vc3$vcov/sum(vc3$vcov), 1)
vc3[, c("grp", "vcov", "pct")]
```

Cross-classification

A level 1 unit can be associated with multiple higher level classifications *jointly*. Examples include:

- Patients measured once, classified by both **hospital** and **neighborhood**

- Ratings classified by both **rater** and **target**

In such cases, the higher-level units are **crossed** rather than nested, with a separate random effect for each classification.

Let y_{ijk} denote the outcome for student i , in school j , and in neighborhood k .

Level 1

$$y_{ijk} = \beta_0 + \beta_1 x_{ijk} + u_{0j} + v_{0k} + e_{ijk}, \quad e_{ijk} \sim \mathcal{N}(0, \sigma^2)$$

Level 2 random effects

$$u_{0j} \sim \mathcal{N}(0, \tau_{\text{school}}^2), \quad v_{0k} \sim \mathcal{N}(0, \tau_{\text{neigh}}^2)$$

Combined

$$y_{ijk} = \beta_0 + \beta_1 x_{ijk} + u_{0j} + v_{0k} + e_{ijk}$$

Additional **variance components** capture heterogeneity:

- τ_{school}^2 (between hospitals)
- τ_{neigh}^2 (between neighborhoods)
- σ^2 (within school–neighborhood combinations)

Here schools and neighborhoods are **independent classifications**, not nested. A patient belongs to one hospital and one neighborhood, but hospitals are not contained within neighborhoods, nor vice versa [1][2].

```
m_cc <- lmer(y ~ x + (1 | school) + (1 | neighborhood), data = df_cc)
```

Extendable to **crossed random slopes** (e.g., school-specific effects of x)

```
m_cc_slope <- lmer(y ~ x + (x | hospital) + (1 | neighborhood), data = df_cc)
```

```
set.seed(22)
n_schools2 <- 4
n_neigh <- 3
n_students2 <- 15

school_u <- rnorm(n_schools2, 0, 8)
neigh_v <- rnorm(n_neigh, 0, 5)

df_cc <- do.call(rbind, lapply(1:n_schools2, function(s) {
  do.call(rbind, lapply(1:n_neigh, function(n) {
```

```
x <- runif(n_students2, 0, 10)
y <- 50 + school_u[s] + neigh_v[n] + 1.5 * x + rnorm(n_students2, 0, 3)
data.frame(x = x, y = y, school = factor(s, labels = paste("School", s)),
           neighborhood = factor(n, labels = paste("Neigh", n)))
}))
}))

ggplot(df_cc, aes(x, y, color = school)) + geom_point(alpha = 0.5, size = 1.8) +
  geom_smooth(method = "lm", se = FALSE, linewidth = 0.9, aes(group = school)) +
  facet_wrap(~neighborhood, nrow = 1) + scale_color_manual(values = rep(pal[c(1,
3, 5)], 4)) + labs(title = "Cross-classified Structure: Students within Schools and Neigh",
  subtitle = "Panels = neighborhoods; colours = schools (crossed, not nested)",
  x = "SES", y = "Test score", color = "School") + guides(color = "none")

vc_cc <- as.data.frame(VarCorr(m_cc))
vc_cc$pct <- round(100 * vc_cc$vcov/sum(vc_cc$vcov), 1)
vc_cc[, c("grp", "vcov", "pct")]

save.image("zenzi.RData")
```

Longitudinal Data

When clustering is because of repeated measurements over time:

- same mixed model dependencies arise within units
- but **time is ordered**.
- temporal structure: measurements closer in time could be more similar

We now re-interpret the two levels:

- **Level 1:** observations (at different time points), within patient
- **Level 2:** patient

The random effects shape individual **trajectories**.

```
dta_timed <- dta_grouped |>
  group_by(group) |>
  arrange(age) |>
  mutate(time = age, timec = factor(1:n())) |>
  select(-age) |>
  ungroup()
```



```
set.seed(123)
.devs <- abs(rnorm(9, 0, 0.5))
.res <- rnorm(64, 0, 2)
dta_timed <- dta_timed |>
  mutate(score_x = score + .devs[group] * time + .res)
# maybe x2 not required
dta_timed_x2 <- dta_grouped_x2 |>
  group_by(group) |>
  mutate(time = age, timec = 1:n()) |>
  select(-age)
set.seed(321)
dta_timed_x2 <- dta_timed_x2 |>
  mutate(score = score + rnorm(n(), 0, 2))
dta_timed_x2 <- dta_timed_x2 |>
  mutate(time_c = scale(time, center = TRUE, scale = FALSE))
```

Categorical and Continuous Time Scale

Time within mixed models

- does not require
 - equidistance
 - fixed timepoints
- can be categorical (ordinal) or continuous (numeric)

Choose continuous when:

- time is meaningful numerically; time elapsed is a driver
- trajectories behave smooth over time

Choose categorical when:

- time is not fully meaningful numerically
 - order can be a driver
 - trajectories change in nature (data generating process)
 - e.g., pre-post, 3m, 6m, 12m

```
| `geom_smooth()` using formula = 'y ~ x'
| `geom_smooth()` using formula = 'y ~ x'
```

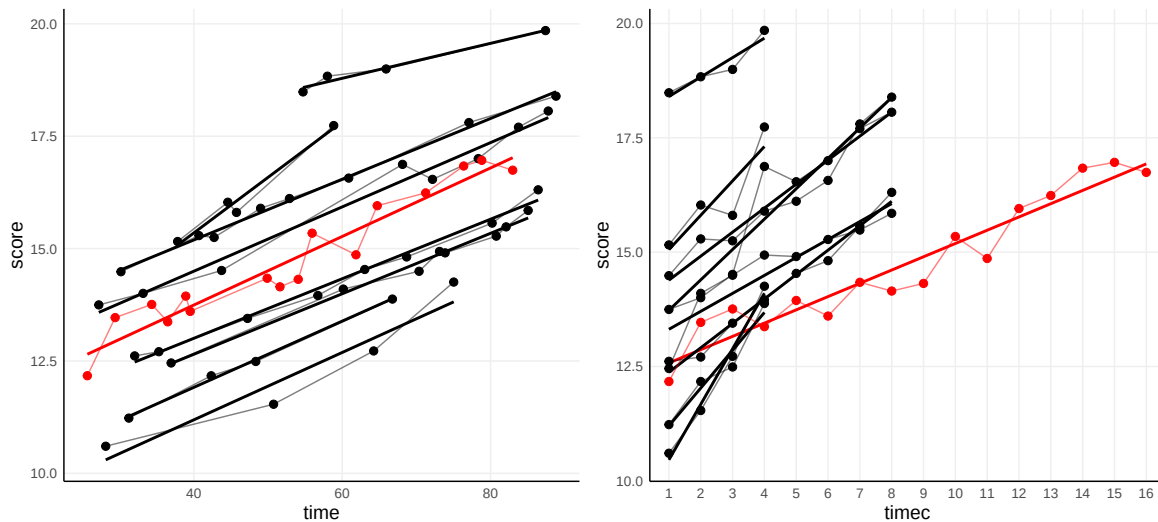


Figure 19: Time Scales: Continuous vs Categorical

The Longitudinal Random Intercept Model

The simplest longitudinal model

- individuals start at different baselines
- change at the same average rate
- just a random intercept model with time as predictor
- correlation within clusters, independent of time difference

$$y_{it} = \underbrace{(\gamma_{00} + u_{0i})}_{\text{person-specific intercept}} + \gamma_{10} \cdot \text{time}_{it} + \varepsilon_{it}$$

with:

$$u_{0i} \sim \mathcal{N}(0, \tau_{00}), \quad \varepsilon_{it} \sim \mathcal{N}(0, \sigma^2)$$

The model fit now uses time as fixed variable to capture the average change, while accomodating a constant covariance between residuals remaining.

```
dta_timed_x <- dta_timed |>
  mutate(score = score_x)
```

```
lmert_int_9 <- lmer(score ~ time + (1 | group), data = dta_timed_x)
```

```
# lmert_int_9 <- lmer(score ~ time + (1 + time | group), data = dta_timed)
pltdta_ri_time <- augment(lmert_int_9, data = dta_timed_x) # |> mutate(group = factor(group))

ranefs_df <- ranef(lmert_int_9)$group |>
  as_tibble(rownames = "group") |>
  mutate(group = factor(group))

# Create group-specific fitted lines (same slope, different intercepts)
pred_data_groups <- ranefs_df |>
  select(group, `(Intercept)`) |>
  crossing(time = seq(min(pltdta_ri_time$time, na.rm = TRUE), max(pltdta_ri_time$time,
    na.rm = TRUE), length.out = 200)) |>
  mutate(fitted = fixef(lmert_int_9)[["(Intercept)"]] + `(Intercept)` + fixef(lmert_int_9)[
    time])

# Population-level fixed effect line
pred_data_pop <- tibble(time = seq(min(pltdta_ri_time$time, na.rm = TRUE), max(pltdta_ri_time
  na.rm = TRUE), length.out = 200)) |>
  mutate(fitted = fixef(lmert_int_9)[["(Intercept)"]] + fixef(lmert_int_9)[["time"]] *
    time)

time_stats <- pltdta_ri_time |>
  summarise(mean_score = mean(score, na.rm = TRUE), se_mean = sd(score, na.rm = TRUE)/sqrt(
    n_groups <- nlevels(pltdta_ri_time$group)
# group_grays_9 <- gray.colors(n_groups, start = 0.25, end = 0.85) col2 <-
# '#5B8FA8'

ggplot(pltdta_ri_time, aes(x = time, y = score)) + geom_point(aes(color = group),
  size = 2.5, alpha = 0.85) + scale_color_manual(values = group_grays_9, guide = "none") +
  geom_segment(aes(x = time, xend = time, y = score, yend = .fitted), color = "gray60",
    alpha = 0.3, linewidth = 0.8) + geom_line(data = pred_data_pop, aes(x = time,
    y = fitted), inherit.aes = FALSE, color = "black", linewidth = 1.2) + geom_line(data = p
    aes(x = time, y = fitted, group = group, color = group), linewidth = 0.9, alpha = 0.7) +
  annotate("text", x = mean(pltdta_ri_time$time, na.rm = TRUE), y = max(pltdta_ri_time$score,
    na.rm = TRUE) + 0.5, label = paste0("score ", round(fixef(lmert_int_9)[["(Intercept)
    2), " + ", round(fixef(lmert_int_9)[["time"]], 3), " × time + random intercept (group)
    size = 4.3) + labs(x = "Time (continuous)", y = "Score", title = "Random Intercept Mo
    subtitle = "Parallel group lines: same slope, different intercepts") + theme_lmm() +
  theme(plot.title = element_text(face = "bold"), plot.subtitle = element_text(size = 11,
    color = "gray40"), panel.grid.minor = element_blank()) + coord_cartesian(xlim = c(min
    na.rm = TRUE), max(pltdta_ri_time$time, na.rm = TRUE) + 0.12 * diff(range(pltdta_ri_time$
```

```
na.rm = TRUE))))
```

Random Intercept Model

Parallel group lines: same slope, different intercepts

$$\text{score} \sim 12.88 + 0.404 \times \text{time} + \text{random intercept}(\text{group})$$

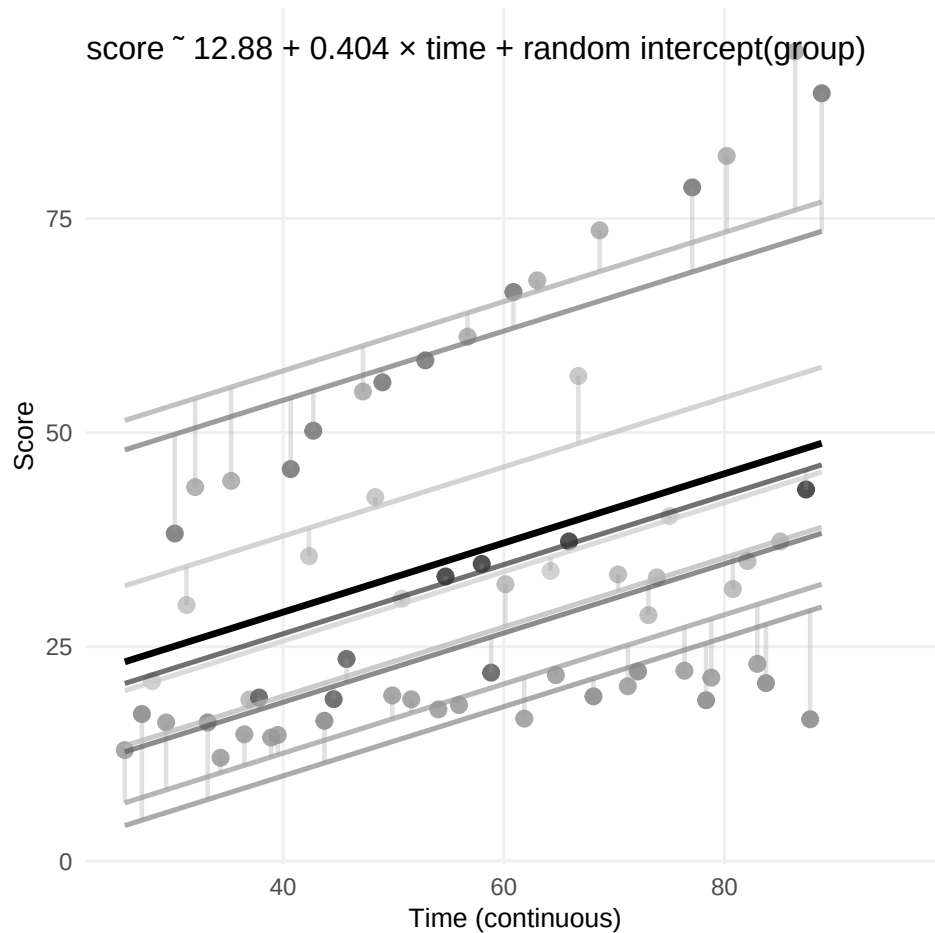


Figure 20: Random intercept over time

Categorical Time

With 4 time points (**T0**, **T3**, **T6**, **T9**), we can compare scores in two ways:

1. **Consecutive timepoints:** T3 vs T0, T6 vs T3, T9 vs T6 (differences between adjacent times)
2. **All vs reference:** T3 vs T0, T6 vs T0, T9 vs T0 (all times compared to baseline)

```
dta_timed_c <- dta_timed |>
  filter(timec %in% c(1, 2, 3, 4))

dta_timed_c$timec <- factor(dta_timed_c$timec, labels = c("T0", "T3", "T6", "T9"),
  levels = c(1, 2, 3, 4))
```

Expected marginal means used to extract contrasts from a fitted model:

```
model_em <- lmer(score ~ timec + (1 | group), data = dta_timed_c)
```

All vs Reference (T0)

```
em_all_vs_ref <- emmeans(model_em, specs = "timec")
tidy(em_all_vs_ref)
| # A tibble: 4 x 6
|   timec estimate std.error    df statistic  p.value
|   <chr>    <dbl>    <dbl> <dbl>    <dbl>    <dbl>
| 1 T0      13.44    0.7319  8.715    18.36 2.874e-8
| 2 T3      14.24    0.7319  8.715    19.45 1.756e-8
| 3 T6      14.61    0.7319  8.715    19.96 1.410e-8
| 4 T9      15.64    0.7319  8.715    21.37 7.867e-9
tidy(contrast(em_all_vs_ref, method = "trt.vs.ctrl", ref = 1))
| # A tibble: 3 x 8
|   term contrast null.value estimate std.error    df statistic  adj.p.value
|   <chr> <chr>      <dbl>    <dbl>    <dbl> <dbl>    <dbl>
| 1 timec T3 - T0      0  0.7989  0.2455 24.00    3.254 0.009422
| 2 timec T6 - T0      0  1.170  0.2455 24.00    4.764 0.0002199
| 3 timec T9 - T0      0  2.201  0.2455 24      8.964 0.00000001179
```

Consecutive Comparisons

```
tidy(contrast(em_all_vs_ref, method = "consec"))
| # A tibble: 3 x 8
|   term contrast null.value estimate std.error    df statistic  adj.p.value
|   <chr> <chr>      <dbl>    <dbl>    <dbl> <dbl>    <dbl>
| 1 timec T3 - T0      0  0.7989  0.2455 24.00    3.254 0.009135
| 2 timec T6 - T3      0  0.3706  0.2455 24.00    1.510 0.3311
| 3 timec T9 - T6      0  1.031  0.2455 24.00    4.200 0.0009479
```

All Pairwise Comparisons

```
contrast(em_all_vs_ref, method = "pairwise")
| contrast estimate SE df t.ratio p.value
| T0 - T3 -0.799 0.245 24 -3.254 0.0166
| T0 - T6 -1.170 0.245 24 -4.764 0.0004
| T0 - T9 -2.201 0.245 24 -8.964 <.0001
| T3 - T6 -0.371 0.245 24 -1.510 0.4475
| T3 - T9 -1.402 0.245 24 -5.710 <.0001
| T6 - T9 -1.031 0.245 24 -4.200 0.0017
|
| Degrees-of-freedom method: kenward-roger
| P value adjustment: tukey method for comparing a family of 4 estimates
```

Custom Comparisons can address specific research questions,
E.g., T9 vs avg(T3, T6) uses contrast parameters 0, -0.5, -0.5, 1

```
contrast(em_all_vs_ref, method = list(`T9 - (T6+T3)/2` = c(0, -0.5, -0.5, 1)))
| contrast estimate SE df t.ratio p.value
| T9 - (T6+T3)/2 1.22 0.213 24 5.722 <.0001
|
| Degrees-of-freedom method: kenward-roger
```

The Longitudinal Random Intercept - Random Slope Model

Heterogeneity can exist over intercept, but also over slopes

- individuals differ in slope
- with an average slope, and an individual specific difference from that average
- time as fixed and random effect

For longitudinal data, individuals can differ in their average but also in their change over time.

$$y_{it} = (\gamma_{00} + u_{0i}) + (\gamma_{10} + u_{1i}) * \text{time}_{it} + \varepsilon_{it}$$

with:

$$\begin{pmatrix} u_{0i} \\ u_{1i} \end{pmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{pmatrix} \tau_{00} & \tau_{01} \\ \tau_{01} & \tau_{11} \end{pmatrix}\right), \quad \varepsilon_{it} \sim \mathcal{N}(0, \sigma^2)$$

The covariance τ_{01} is often negative, starting higher results in stronger decrease, or starting lower in stronger increase.

effect	group	Term	Estimate	Std. Error	t value	df	p value
fixed	NA	(Intercept)	11.227	1.142	9.830	6.740124	3.07×10^{-5}
fixed	NA	time	0.438	0.105	4.171	8.078695	3.05×10^{-3}
ran_pars	group	sd__(Intercept)	2.005	NA	NA	NA	NA
ran_pars	group	cor__(Intercept).time	-0.125	NA	NA	NA	NA
ran_pars	group	sd__time	0.312	NA	NA	NA	NA
ran_pars	Residual	sd__Observation	1.908	NA	NA	NA	NA

Again: To avoid convergence issues you should center the random slope to reduce colinearity with the intercept.

```
lmert_slp_9 <- lmer(score ~ time + (1 + time | group), data = dta_timed_x)
```

```
tidy_gt(lmert_slp_9)
```

```
# lmert_int_9 <- lmer(score ~ time + (1 + time | group), data = dta_timed)
pltdta_rs_time <- augment(lmert_slp_9, data = dta_timed_x) # |> mutate(group = factor(group))

ranefs_df <- ranef(lmert_slp_9)$group |>
  as_tibble(rownames = "group") |>
  mutate(group = factor(group))

# Create group-specific fitted lines (same slope, different intercepts)
pred_data_groups <- ranefs_df |>
  select(group, `(Intercept)`, rtime = time) |>
  crossing(time = seq(min(pltdta_rs_time$time, na.rm = TRUE), max(pltdta_rs_time$time,
    na.rm = TRUE), length.out = 200)) |>
  mutate(fitted = fixef(lmert_slp_9)[["(Intercept)"]] + `(Intercept)` + (fixef(lmert_slp_9)[["time"]] *
    rtime) * time)

# Population-level fixed effect line
pred_data_pop <- tibble(time = seq(min(pltdta_rs_time$time, na.rm = TRUE), max(pltdta_rs_time$time,
  na.rm = TRUE), length.out = 200)) |>
  mutate(fitted = fixef(lmert_slp_9)[["(Intercept)"]] + fixef(lmert_slp_9)[["time"]] *
    time)

time_stats <- pltdta_rs_time |>
  summarise(mean_score = mean(score, na.rm = TRUE), se_mean = sd(score, na.rm = TRUE)/sqrt(
    n_groups <- nlevels(pltdta_rs_time$group)
```

```
# group_grays_9 <- gray.colors(n_groups, start = 0.25, end = 0.85) col2 <-  
# '#5B8FA8'
```

```
ggplot(pltdta_rs_time, aes(x = time, y = score)) + geom_point(aes(color = group),  
  size = 2.5, alpha = 0.85) + scale_color_manual(values = group_grays_9, guide = "none") +  
  geom_segment(aes(x = time, xend = time, y = score, yend = .fitted), color = "gray60",  
    alpha = 0.3, linewidth = 0.8) + geom_line(data = pred_data_pop, aes(x = time,  
  y = fitted), inherit.aes = FALSE, color = "black", linewidth = 1.2) + geom_line(data = p  
  aes(x = time, y = fitted, group = group, color = group), linewidth = 0.9, alpha = 0.7) +  
  annotate("text", x = mean(pltdta_rs_time$time, na.rm = TRUE), y = max(pltdta_rs_time$score,  
    na.rm = TRUE) + 0.5, label = paste0("score  ", round(fixef(lmert_slp_9)[["(Intercept)",  
      2), " + ", round(fixef(lmert_slp_9)[["time"]], 3), " × time + random effects(group)".  
    size = 4.3) + labs(x = "Time (continuous)", y = "Score", title = "Random Intercept, I  
  subtitle = "Non-parallel group lines: different slopes, different intercepts") +  
  theme_lmm() + theme(plot.title = element_text(face = "bold"), plot.subtitle = element_text  
  color = "gray40"), panel.grid.minor = element_blank()) + coord_cartesian(xlim = c(min(pl  
  na.rm = TRUE), max(pltdta_rs_time$time, na.rm = TRUE) + 0.12 * diff(range(pltdta_rs_time  
  na.rm = TRUE))))
```


Random Intercept, Random Slope Model

Non-parallel group lines: different slopes, different intercepts

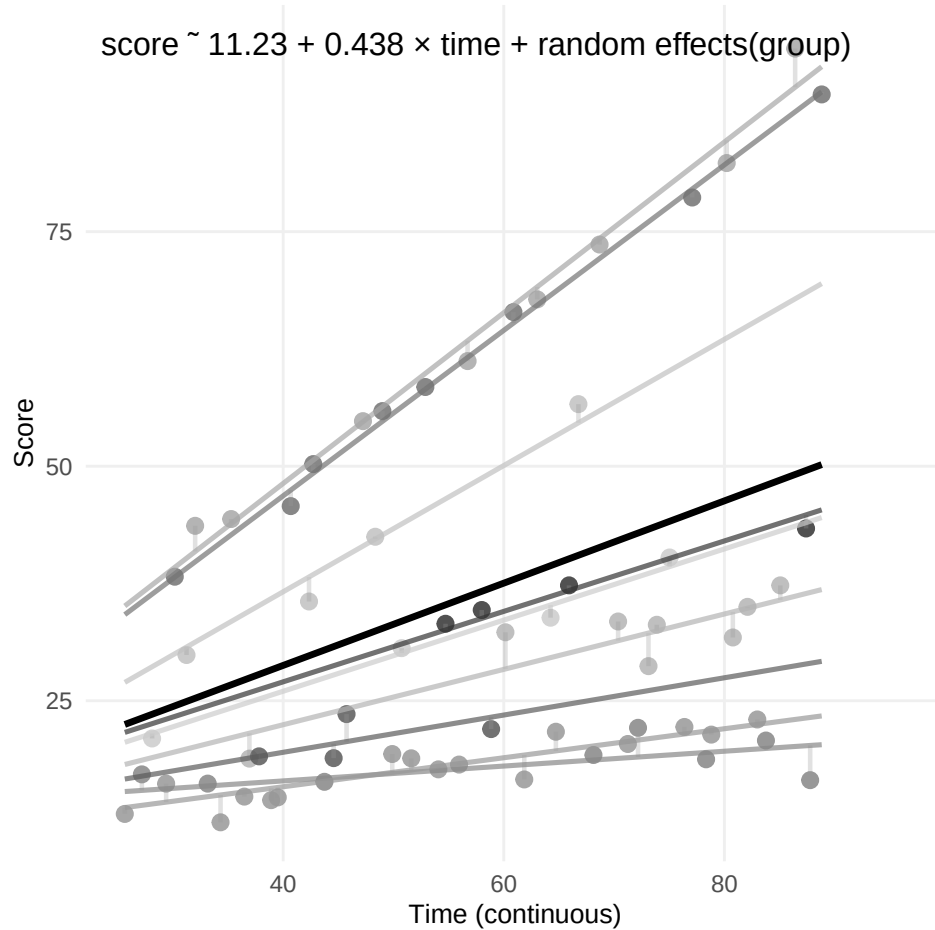


Figure 21: Random slope model over time

Covariance Structures

Within cluster correlational structures:

- correlation same for all pairs
- correlation different for all pairs
- correlation structured (in between the above)

Choose **covariance structure** for the outcome vector $\mathbf{y}_i$ of patient i .

Note: covariance structures

- implied by the variance-covariance estimated
- specified as a repeated measures structure (residual covariance).

For $T = 4$ time points, the covariance matrix Σ_i is 5×5 .

Most Common Structures

Compound Symmetry (CS)

All variances equal; all covariances equal regardless of time lag.

$$\Sigma_{CS} = \begin{pmatrix} \sigma^2 & \rho\sigma^2 & \rho\sigma^2 & \rho\sigma^2 \\ \rho\sigma^2 & \sigma^2 & \rho\sigma^2 & \rho\sigma^2 \\ \rho\sigma^2 & \rho\sigma^2 & \sigma^2 & \rho\sigma^2 \\ \rho\sigma^2 & \rho\sigma^2 & \rho\sigma^2 & \sigma^2 \end{pmatrix}$$

This is exactly what a random intercept model implies. Two parameters: σ^2 and ρ .

Note: this is often used for clustered data, not necessarily repeated measurements.

Unstructured (UN)

No constraints — every variance and every covariance is estimated freely. For T time points: $T(T + 1)/2$ parameters. Most flexible, but most expensive in terms of parameters.

$$\Sigma_{Unstructured} = \begin{pmatrix} \sigma_1^2 & \sigma_{12} & \sigma_{13} & \sigma_{14} \\ \sigma_{12} & \sigma_2^2 & \sigma_{23} & \sigma_{24} \\ \sigma_{13} & \sigma_{23} & \sigma_3^2 & \sigma_{34} \\ \sigma_{14} & \sigma_{24} & \sigma_{34} & \sigma_4^2 \end{pmatrix}$$

Heterogeneous Compound Symmetry (CSH)

Like CS, but allows variances to differ across time points. Useful when variability grows over follow-up.

$$\Sigma_{CSH} = \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 & \rho\sigma_1\sigma_3 & \rho\sigma_1\sigma_4 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 & \rho\sigma_2\sigma_3 & \rho\sigma_2\sigma_4 \\ \rho\sigma_1\sigma_3 & \rho\sigma_2\sigma_3 & \sigma_3^2 & \rho\sigma_3\sigma_4 \\ \rho\sigma_1\sigma_4 & \rho\sigma_2\sigma_4 & \rho\sigma_3\sigma_4 & \sigma_4^2 \end{pmatrix}$$

Autoregressive Order 1 — AR(1)

Correlation decays exponentially with lag: $\text{Corr}(y_t, y_{t+k}) = \rho^k$.

$$\Sigma_{\text{AR}(1)} = \sigma^2 \begin{pmatrix} 1 & \rho & \rho^2 & \rho^3 \\ \rho & 1 & \rho & \rho^2 \\ \rho^2 & \rho & 1 & \rho \\ \rho^3 & \rho^2 & \rho & 1 \end{pmatrix}$$

Two parameters: σ^2 and ρ . Natural for equally-spaced measurements with decaying memory.

Heterogeneous AR(1)

Allow the variances to change over time.

$$\Sigma_{\text{ARH}(1)} = \begin{pmatrix} \sigma_1^2 & \rho \sigma_1 \sigma_2 & \rho^2 \sigma_1 \sigma_3 & \rho^3 \sigma_1 \sigma_4 \\ \rho \sigma_1 \sigma_2 & \sigma_2^2 & \rho \sigma_2 \sigma_3 & \rho^2 \sigma_2 \sigma_4 \\ \rho^2 \sigma_1 \sigma_3 & \rho \sigma_2 \sigma_3 & \sigma_3^2 & \rho \sigma_3 \sigma_4 \\ \rho^3 \sigma_1 \sigma_4 & \rho^2 \sigma_2 \sigma_4 & \rho \sigma_3 \sigma_4 & \sigma_4^2 \end{pmatrix}$$

Toeplitz

Correlations depend only on the lag, but without the exponential constraint of AR(1). Each lag distance has its own parameter. More flexible than AR(1), fewer parameters than unstructured.

$$\Sigma_{\text{Toeplitz}} = \begin{pmatrix} \tau_0 & \tau_1 & \tau_2 & \tau_3 \\ \tau_1 & \tau_0 & \tau_1 & \tau_2 \\ \tau_2 & \tau_1 & \tau_0 & \tau_1 \\ \tau_3 & \tau_2 & \tau_1 & \tau_0 \end{pmatrix}$$

6. ...

Various others exist, choose

- most efficient
- captures the assumed process
 - continuous or not, equidistant or not ?
- is in line with the data

- choose based on AIC or BIC (fitted with ML)

```
save.image("zenzi.RData")
```

Final Remarks

Mixed models

- are regression models, extended with a model for dependencies
- explicitly model two (or more) sources of variation:
 - within-cluster variation
 - between-cluster variation
- use clusters (e.g., patients, hospitals, sites), linked through a common population of clusters
- capture cluster-specificities through:
 - random effects (cluster-specific deviations)
 - induced correlation structures for repeated/linked measurements
- allow borrowing strength across clusters via partial pooling
- provide flexible covariance structures for errors / random effects
 - e.g., AR(1), heterogeneous AR(1), compound symmetry, heterogeneous CS, unstructured, Toeplitz
- yield more accurate inference for:
 - fixed effects (regression coefficients)
 - predicted cluster-specific trajectories / means
- handle unbalanced data and missing observations naturally
- are appropriate for:
 - repeated measures, longitudinal data
 - hierarchical / nested data
 - clustered survey data, multi-site studies

welcome to discuss further @ SQUARE book an appointment through square.research.vub.be

```
save.image("zenzi.RData")
```